



TEKNOLOJİ FAKÜLTESİ İŞLETMEDE MESLEKİ EĞİTİM RAPORU

Adı Soyadı : Murat BURÇ
Bölümü : Elektrik Elektronik Mühendisliği
İşletmenin Adı : AISOFT Yazılım Şirketi
Denetçi Öğretim Elemanı : Doç. Dr. Ali Furkan KAMANLI
Öğretim Yılı ve Dönemi : 2024-2025 Bahar Yarıyılı

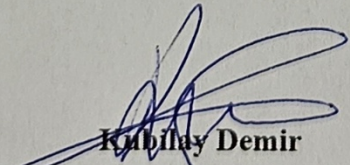


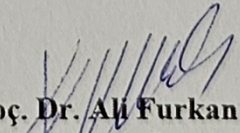
T.C.
SAKARYA UYGULAMALI BİLİMLER ÜNİVERSİTESİ
TEKNOLOJİ FAKÜLTESİ
İŞLETMEDE MESLEKİ EĞİTİM FİNAL RAPORU



Öğrencinin Adı Soyadı	: Murat BURÇ
Bölümü	: Elektrik Elektronik Mühendisliği
Numarası	: B170900008
İşletmenin Adı	: AISOFT Yazılım Şirketi
İşletmede Mesleki Eğitim Sorumlusu	: Kubilay DEMİR
Denetçi Öğretim Elemanı	: Doç. Dr. Ali Furkan KAMANLI
Öğretim Yılı ve Dönemi	: 2024-2025 Bahar Yarıyılı

*Bu 7+1 İşletmede Mesleki Eğitim Final Raporu/...../..... tarihinde
aşağıdaki işletmede mesleki eğitim sorumluları tarafından kabul edilmiştir*


Kubilay Demir
İşletmede Mesleki Eğitim Sorumlusu


Doç. Dr. Ali Furkan KAMANLI
Denetçi Öğretim Elemanı

ÖNSÖZ

7+1 İşyeri uygulaması kapsamında hazırlanan bu ara rapor, 9 hafta boyunca işyeri uygulaması kapsamında yapılan çalışmaları ve elde edilen deneyimleri içermektedir. Rapordaki konular uygulama boyunca süren çalışmalardan oluşmaktadır. 9 haftalık bir değerlendirme sonucu elde edilen deneyimleri ortaya koymaktadır.

Çalışmalarım sırasında büyük desteklerini gördüğüm İşyeri Eğitim Sorumlusu Sn. Kubilay Demir'e ve katkılarından dolayı Sn. Yusuf Cihan Gedik'e teşekkürlerimi sunarım.

Sakarya, 2025

Murat BURÇ

İÇİNDEKİLER

ÖNSÖZ	ii
İÇİNDEKİLER	iii
SİMGE VE KISALTMALAR	vii
ŞEKİLLER LİSTESİ	viii
TABLolar LİSTESİ	x
ÖZET	xi
SUMMARY	xii

BÖLÜM 1.

GİRİŞ	1
1.1 İşyeri Tanıtımı	1
1.1.1. AISOFT MES+	1
1.1.2. AISOFT SAFE	2
1.1.3. AISOFT Quality	2
1.1.4. AISOFT 2dCheck	3
1.1.5. AISOFT CCR	3
1.1.6. AISOFT MIL	4

BÖLÜM 2.

SÜREÇ YÖNETİMİ	5
2.1. İşletmedeki İdari Süreçler	5
2.2 İşletmedeki Üretim Süreçleri	5

BÖLÜM 3.

STANDARDİZASYON	7
3.1. Genel Mühendislik Standartları	7
3.1.1. ISO(International Standards Organization)	7
3.1.1.1. ISO standart oluşturma aşamaları	7
3.1.1.2. Başlıca ISO standartları	8

3.1.2. IEC(International Electrotechnical Commission)	9
3.1.2.1. IEC başlıca yapay zeka standartları	10
3.2. İşletmede Uygulanan Standartlar	10
3.3. İşletmedeki Kalite Yönetimi Sistemi	11
3.3.1. Kalite yönetim süreci aşamaları	12
3.3.1.1. Planlama	12
3.3.1.2. Uygulama	12
3.3.1.3. Kontrol	12
3.3.1.4. Önlem ve iyileştirme	12
BÖLÜM 4.	
İŞYERİ UYGULAMALARI	13
4.1. Kaynak Kodu Güvenliği	13
4.1.1. Docker	13
4.1.1.1. Docker çalışma mantığı	13
4.1.1.2. Docker'ın temel özellikleri	14
4.1.3. Problem, sunulan çözümler ve karşılaşılan zorluklar	14
4.3.3.1. Hukuki yöntem	15
4.3.3.2. Docker imajı içinde kaynak kod şifreleme	15
4.3.3.3. Docker build sürecinde kaynak kodun sadece geçici olarak kullanılması	15
4.3.3.4. Read-only(salt okunur) volume ile kodun değiştirilememesi	16
4.3.3.5. Derlenmiş binary dosyalarının kullanımı	16
4.3.3.6. Distroless veya scratch image kullanımı	16
4.3.3.7. Kod obfuscation(karmaşılaştırma) ile anlaşılabilir hale getirme	17
4.3.3.8. Bellekte çalışan kodun iz bırakmadan yürütülmesi.	17
4.3.3.9. Lisans sunucu doğrulaması ile yetkisiz çalışmanın engellenmesi	17
4.3.3.10. AppArmor/SELinux/seccomp ile sistem erişimini sınırlama	18
4.3.3.11 Docker secrets ile hassas bilgilerin korunması	18
4.3.3.12. Shell ve debug araçlarının konteynerden kaldırılması ...	18

4.3.3.13. Anti-Tamper mekanizmaları	18
4.3.3.14. TPM/SGX gibi donanım tabanlı güvenlik sistemleriyle entegrasyon	19
4.3.3.15. Kodun API'den alınarak RAM'de çalıştırılması	19
4.3.3.16. Özel EntryPoint script ile ortam doğrulama	19
4.3.3.17 Özel ve şifreli Docker registry kullanımı	19
4.3.3.18. Şifrelenmiş dosya sistemi(LUKS, eCryptFS) kullanımı	19
4.3.3.19. Docker layer temizliği ve minimal image üretimi	20
4.3.3.20. Runtime ortamda ağ bağlantısı olmadan çalışacak şekilde tasarlama	20
4.2 Veri Etiketleme	20
4.2.1. Veri etiketleme yöntemleri	21
4.2.1.1 Manuel etiketleme	21
4.2.1.2. Yarı otomatik etiketleme	21
4.2.1.3. Otomatik etiketleme	22
4.2.1.4 Aktif öğrenme	22
4.2.2. Yaygın veri etiketleme formatları	22
4.2.2.1. YOLO TXT	22
4.2.2.2. Pascal VOC	23
4.2.2.3. LabelME JSON	24
4.3. HIKROBOT Kamera Python Entegrasyonu	24
4.3.1. HIKROBOT markası ve kullanılan kameralar	25
4.3.1.1 GigE Vision	25
4.3.1.2. 6-pin Hirose I/O konektörü	26
4.3.2 Machine vision software development kit	26
4.3.3. Kamera ile iletişimde kullanılan kod	27
BÖLÜM 5.	
KARMAŞIK MÜHENDİSLİK UYGULAMASI.....	31
5.1. Genel Bilgiler	31
5.2. Yapılandırmalar	32
5.3. PLC ile Haberleşme - Arayüz Entegrasyonu	34
5.3.1. PLC Nedir.....	34
5.3.2. Modbus.....	35

5.3.3. TCP.....	35
5.3.4. Modbus TCP.....	35
5.3.5. Pymodbus kütüphanesi.....	36
5.3.6. Retro Reflektif Fotoelektrik Sensör	37
5.3.7. Yapılan İşlemler	38
BÖLÜM 6.	
DİSİPLİNLER ARASI UYGULAMALAR	47
BÖLÜM 7.	
İŞLETMEDE KARŞILAŞILAN PROBLEMLER VE ÇÖZÜMLER.....	48
BÖLÜM 8.	
SONUÇ ve DEĞERLENDİRME	49

SİMGE VE KISALTMALAR

API	:Application Programming Interface
GigE	: Gigabit Ethernet
IEC	: International Electrotechnical Commission
ISO	: International Organization for Standardization
JSON	: JavaScript Object Notation
LUKS	: Linux Unified Key Setup
MVS	: Machine Vision Software
MVS SDK	: Machine Vision Software Software Development Kit
Pascal VOC	: Pattern Analysis, Statistical Modelling and Computational Learning Visual Object Classes
PoE	: Power over Ethernet
SGX	: Software Guard Extensions
TPM	: Trusted Platform Module
YOLO	: You Only Look Once
CAD	: Computer-Aided Design

ŞEKİLLER LİSTESİ

Şekil 1.1. AISOFT Logo	1
Şekil 1.2. AISOFT MES+ arayüz	2
Şekil 1.3. AISOFT Safe arayüz	2
Şekil 1.4. AISOFT Quality tanıtım örneği	3
Şekil 1.5. AISOFT 2dCheck tanıtım örneği	3
Şekil 1.6. AISOFT CCR uygulama örneği	4
Şekil 1.7. AISOFT MIL tanıtım örneği	4
Şekil 2.1. Organizasyon şeması	5
Şekil 3.1. ISO logo	7
Şekil 3.2. ISO 9000 logo	8
Şekil 3.3. ISO 9001:2015 logo	11
Şekil 4.1. Docker logo	13
Şekil 4.2. Docker engine parçaları	14
Şekil 4.3. Multi-Stage build ve binarye çevirme	16
Şekil 4.4. Kod karmaşıklıklaştırma	17
Şekil 4.5. Etiketleme örneği	21
Şekil 4.6. Yapay zeka destekli etiketleme örneği	21
Şekil 4.7. YOLO txt dosyası örneği	22
Şekil 4.8. Pascal VOC XML örneği	23
Şekil 4.9. LabelMe JSON formatı örneği	24
Şekil 4.10. HIKROBOT MVS kamera ve etiket bilgileri	25
Şekil 4.11. HIKROBOT Kamera 6-Pin hirose ve PoE girişi	26
Şekil 4.12. Tetikleme modu kod bloğu	27
Şekil 4.13. Yükselen kenar gösterimi	28
Şekil 4.14. Kamera kontrol sürekli çekim(freerun) kod bloğu	28
Şekil 4.15. Kamera kontrol tetik bekleme kod bloğu	29
Şekil 4.16. Fotoğraf çekme ve kamera kapatma kod bloğu	30
Şekil 5.1. Python özel modül import edilmesi	32
Şekil 5.2. PLC CLI arayüzünün kontrol kısmı.....	32
Şekil 5.3. Cam ölçüm arayüzü	33
Şekil 5.4. Retro reflektif fotoelektrik sensör	37
Şekil 5.5. PLC kontrolünde Modbus yardımcı fonksiyonları	38
Şekil 5.6. PyQt thread işlemleri için QThread sınıfından miras alma	38
Şekil 5.7. JSON dosyasının yüklenmesi	39
Şekil 5.8. PLC config JSON dosyasından bazı register adresleri	39
Şekil 5.9. JSON dosyasından adreslerin alınması	39
Şekil 5.10. PLC için kullanılan fonksiyonlardan bir örnek.....	40

Şekil 5.11. Pymodbus kütüphanesi write_coil fonksiyonu	40
Şekil 5.12. CAD dosya yolları	42
Şekil 5.13. PyQt CAD listesi yükle butonu.....	42
Şekil 5.14. retranslateUi fonksiyonu ile buton isimlendirmesi	43
Şekil 5.15. CAD listesi için kullanılacak değişkenler	43
Şekil 5.17. Full otomatik buton sinyaline fonksiyon bağlanması	43
Şekil 5.18. CAD listesi yükleme fonksiyonu	44
Şekil 5.19. load_cad_list fonksiyonu için yardımcı fonksiyon	45
Şekil 5.20. Full otomatik ölçüm fonksiyonu	45
Şekil 5.21. Sinyal tetiklemesine göre CAD değişimi.....	46

TABLÖLAR LİSTESİ

Tablo 3.1 Başlıca Standartlar	10
Tablo 4.1 – HIKROBOT Kamera özellikleri	25

ÖZET

7+1 İşyeri Uygulamasına 24/02/2025 tarihinde AISOFT Yazılım Anonim Şirketinde başlanmış ve AR-GE ofisinde görev alınmıştır. Bu ara rapor 9 haftalık çalışma süresini kapsamaktadır. Yapay zekâ modellerinin eğitilmesi amacıyla veri setleri doğru ve etkin bir şekilde etiketlenmiştir. Otomatik veri etiketleme araçları araştırılmış, test edilmiş ve uygun olan araçlar belirlenmiştir. Görüntü segmentasyonu projelerinde hassas etiketleme çalışmaları yapılmıştır. Çeşitli yapay zekâ modellerinin eğitimleri gerçekleştirilmiş ve performans analizleri yapılmıştır. Ayrıca Docker teknolojisi üzerine uygulamalı çalışmalar yapılmış, Docker konteynerlerinde kaynak kodunun güvenliğine yönelik araştırmalar ve uygulamalar gerçekleştirilmiştir. Bu süreç boyunca elde edilen bilgiler ve deneyimler, teorik bilgilerin pratiğe dökülmesi ve profesyonel çalışma ortamının deneyimlenmesine imkân sağlamıştır.

SUMMARY

The 7+1 Workplace Practice began on 24/02/2025 at AISOFT Software Incorporated Company, and duties were carried out in the R&D office. This interim report covers a 9-week working period. In order to train artificial intelligence models, datasets were labeled accurately and efficiently. Automatic data labeling tools were researched, tested, and the most suitable tools were identified. Precise labeling tasks were conducted for image segmentation projects. Various AI models were trained, and performance analyses were performed. In addition, hands-on studies were carried out on Docker technology, including research and implementations focused on source code security within Docker containers. The knowledge and experience gained throughout this process provided the opportunity to put theoretical knowledge into practice and to gain insight into a professional working environment.

BÖLÜM 1. GİRİŞ

AISOFT, endüstri için bilgisayarlı görü çözümleri sunan bir şirkettir. Gerek geleneksel bilgisayarlı görü teknikleri gerekse makine öğrenmesi/derin öğrenme temelli bilgisayarlı görü yöntemleri ile çeşitli alanlarda hizmet vermektedir. Endüstri 4.0 uyumlu sistemlerle üretim süreçlerini optimize ederek maliyetlerin azaltılmasını ve kalitenin artırılmasını amaçlar.

1.1 İşyeri Tanıtımı

AISOFT yazılım anonim şirketi endüstriyel işletmelerin verimlilik ve kalite hedeflerine ulaşmalarını sağlamak için yapay zeka ve bilgisayarlı görü tabanlı çözümler sunar. 7 kişilik bir çalışma kadrosu bulunmaktadır. Elektronik Haberleşme Mühendisi Kubilay Demir ve Elektronik Haberleşme Mühendisi Selahattin Koşunalp ve Vedat Tümen tarafından Mart 2022’de kurulmuştur. AISOFT Yazılım Anonim Şirketi, İstiklal Mahallesi, Çark Caddesi, 291/108, H. Recep Tan İş Merkezi Kat:1, No:14, 54050, Serdivan/Sakarya adresinde bulunmaktadır



Şekil 1.1. AISOFT Logo

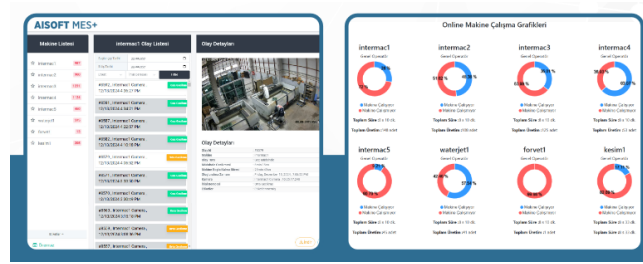
1.1.1. AISOFT MES+

AISOFT MES+ sistemi, üretim süreçlerini gerçek zamanlı olarak izleyerek üretim miktarlarını ve meydana gelen üretim gecikmelerini eşzamanlı şekilde raporlayabilmektedir. Bu sistem sayesinde üretim hattında meydana gelen aksamaların sebepleri hızlıca analiz edilmekte ve üretim verimliliği kaybı minimum seviyeye indirilmektedir. Anlık bildirim özelliği, üretim yöneticilerinin süreçten haberdar olmasını sağlayarak hızlı karar almalarına imkân tanımaktadır.

MES+ yazılımı, üretim hattında oluşabilecek aksaklıkları yapay zekâ destekli algoritmalar aracılığıyla tespit etmekte ve bu aksaklıkları doğrudan ilgili yöneticiye bildirmektedir. Bu sayede, olası üretim duraksamaları ve kalite kayıplarına karşı zamanında müdahale edilerek üretim sürekliliği sağlanmaktadır. Erken uyarı

mekanizmaları ile donatılmış olan bu sistem, üretimde yaşanabilecek kayıpları önlemeye yönelik etkili bir çözüm sunmaktadır.

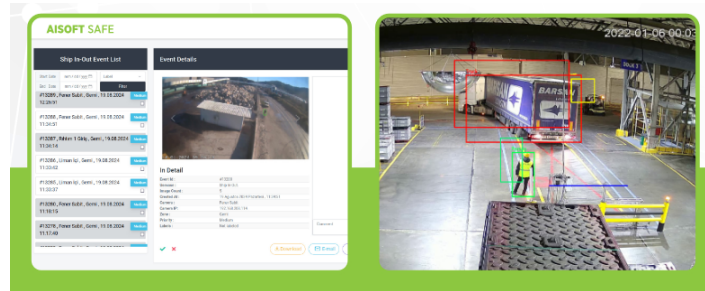
Geçmişe dönük üretim verilerinin detaylı analizi AISOFT MES+ sistemi ile gerçekleştirilebilmektedir. Makine performansı, operatör verimliliği ve üretim hatlarındaki genel eğilimler sistem tarafından analiz edilmekte ve yöneticilere kapsamlı raporlar halinde sunulmaktadır. Bu raporlar sayesinde işletmeler, gelecekteki üretim planlarını daha verimli hale getirebilmekte ve sürekli iyileştirme süreçlerine veri temelli katkılar sağlayabilmektedir.



Şekil 1.2. AISOFT MES+ arayüz

1.1.2. AISOFT Safe

AISOFT SAFE, iş sağlığı ve güvenliği ihlallerini yapay zekâ destekli sistemlerle otomatik olarak tespit eder, ilgili birimlere bildirir ve detaylı olarak raporlar. İnsan inisiyatifine dayalı kontrolleri ortadan kaldırarak çalışanların kurallara tam uyum göstermesini sağlar. Sistem, tespit ettiği ihlalleri anlık olarak ilgili birimlere ileterek İSG süreç otomasyonunu sağlar. Aynı zamanda, süreçlerin kesintisiz izlenmesini mümkün kılar ve ihlallere ilişkin çözüm önerilerini güçlü raporlama altyapısıyla doküman eder. Bu sayede, güvenli ve sürdürülebilir bir çalışma ortamı oluşturulmasına katkı sağlar.

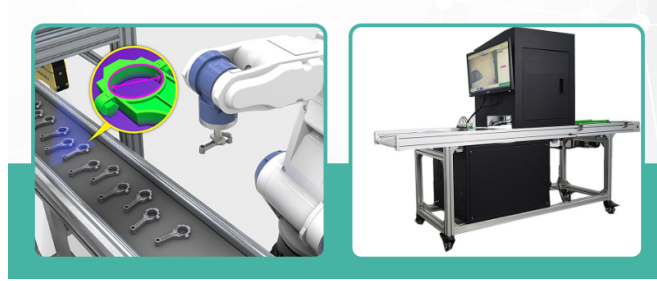


Şekil 1.3. AISOFT Safe arayüz

1.1.3. AISOFT Quality

AISOFT QUALITY, üretim sürecinde ortaya çıkabilecek anormallikleri yapay zekâ ile erken aşamada tespit ederek hataların yayılmasını önler ve müşteriye hatalı ürün teslim edilmesini engeller. Üretimin her aşamasını detaylı biçimde kontrol altında

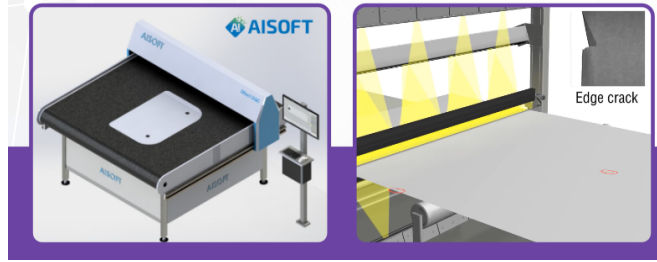
tutarak hem öncesi hem sonrası süreçleri izler. Belirlenen sapmalara karşı proaktif müdahale olanağı sunan sistem, üretim hattına doğrudan ve hızlı şekilde müdahale etmeye imkân tanır. Kullanıcı dostu arayüzü ve kolay kurulum özellikleriyle, operatörler tarafından rahatlıkla kullanılabilir bir yapıdadır.



Şekil 1.4. AISOFT Quality tanıtım örneği

1.1.4. AISOFT 2dCheck

TÜBİTAK desteğiyle geliştirilen AISOFT 2dCheck, iki boyutlu nesnelerin yüzey kalitesini ve boyutsal ölçümlerini yüksek hassasiyetle denetleyen gelişmiş bir kontrol sistemidir. CAD çizimleri ile karşılaştırmalı analiz yaparak üretim doğruluğunu hassas biçimde kontrol eder. Yüzeydeki tüm kusurları tespit edip operatör ekranına yansıtarak görsel uyarı sağlar. Anlık müdahale imkânı sayesinde kusurlu ürünlerin zamanında ayrıştırılmasına olanak tanır ve kaliteyi garanti altına alır.



Şekil 1.5. AISOFT 2dCheck tanıtım örneği

1.1.5. AISOFT CCR

AISOFT CCR, ECE R-43 standartlarına uygun olarak temperli camların kırılma testlerini otomatikleştiren bir sistemdir. Kırılmış cam parçalarını 5x5 cm'lik alanlarda tarayarak hem parça sayısını hem de en büyük parçayı ölçer. Detaylı raporlama özelliği sayesinde maksimum ve minimum ölçüler firma bilgileriyle birlikte kayıt altına alınır. Elde edilen veriler hem yerel veri tabanında hem de bulut ortamında saklanabilir; talep halinde e-posta yoluyla ilgili birimlere iletilir. Bu sayede test süreci hem hızlı hem de izlenebilir hale gelir.



Şekil 1.6. AISOFT CCR uygulama örneği

1.1.6. AISOFT MIL

AISOFT MIL, insanlı ve insansız araçların çevresinde bulunan hareketli unsurları tespit ve takip etmek üzere geliştirilmiş yapay zekâ tabanlı bir izleme sistemidir. Kısmen gizlenmiş ya da tam görünmeyen nesneleri yapay zekâ algoritmalarıyla algılayarak üstün bir farkındalık sağlar. Yüksek performansı sayesinde mini bilgisayarlar üzerinde bile sorunsuz şekilde çalışabilir. İnsan ve araç sayımı, konum takibi ve genel durum değerlendirmesi gibi çok amaçlı kullanım senaryolarına uygundur.



Şekil 1.7. AISOFT MIL tanıtım örneği

BÖLÜM 2. SÜREÇ YÖNETİMİ

AISOFT Yazılım Anonim Şirketi firma/proje bazlı çalışmakta ve kaynaklarını/çalışanları buna göre bölümlendirmektedir ancak gereken noktalarda bütün çalışanlar bütün projelerde bilgisi dahilinde yetki sahibidir. Startup firması olmasının da getirdiği çalışma durumu ile birlikte çalışanlar bütün projelerde belirli seviyelerde bilgi ve yetki sahibidir.

2.1. İşletmedeki İdari Süreçler



Şekil 2.1. Organizasyon şeması

2.2. İşletmedeki Üretim Süreçleri

AISOFT yazılım anonim şirketi, bilgisayarlı görü, yapay zeka tabanlı yazılım sistemleri geliştiren bir Ar-Ge firmasıdır. Firma üretim süreci klasik bir fiziksel üretim sürecinden daha çok yazılım geliştirme döngüsüne uyularak yönetilmektedir. Bu çerçevede üretim süreçleri;

- Müşteri taleplerinin alınması
- Taleplerin analiz edilmesi ve yöntem seçimi
- Birincil ihtiyaçların belirlenmesi
- Planlama
- Yapay zeka modelleme ve test
- Yazılım ve donanım geliştirme
- Test
- Doğrulama ve bakım aşamalarından oluşmaktadır

Bütün süreçler detaylı şekilde dokümanite edilmekte ve proje devamlılığı garanti altına alınmaktadır. Her projede müşteri gereksinimleri belirlenmekte, ardından bu taleplere

özüm olabilecek yöntemler listelenmekte ve en uygun yöntem seçilmektedir. Bu aşamadan sonra yol haritası çıkarılmaktadır. Bundan sonra yapay zeka modelleme kısmında işe uygun veriler toplanmakta, hassas bir şekilde etiketlenme işlemleri gerçekleştirilmektedir. Ardından seçilen model eğitilerek test edilmektedir. Projeye uygun yapay zeka modeli üretim sürecinin başlarında talep analizi kısmında belirleniyor olsa da modelin senaryoya uygun performansı verememesi veya daha iyi sonuç alınabileceği düşüncesi durumunda model değişikliğine gidilebilmektedir bunun yanı sıra eğer mümkünse sürekli kaliteli veri takviyesi yapılmaktadır. Yapay zeka modelleme aşamasının yanı sıra yazılım ve donanım geliştirme aşaması da model ile birlikte sürdürölmektedir. Projenin ihtiyaçlarına uygun kameranın seçilmesi ve bu kamera için gerekli elektronik ve yazılım altyapısı hazırlanmaktadır. Raporun ilerleyen süreçlerinde örnek verileceği üzere cam kırıklarını sayan yapay zeka modelinin kullandığı kameranın tetiklenmesi bu elektronik butonun yazılımla iletişime geçmesi, kameranın aldığı görüntülerin yazılımda yapay zeka modeli ile işlenmesi ve bu işlemlerin GUI aracılığıyla takip edilmesi, yazılım ve donanım geliştirme aşamasına verilebilecek örneklerdendir. Test aşaması hem yapay zeka modelleme hem de yazılım ve donanım geliştirme aşamalarında sürekli yapılan bir konu olsa da bu iki aşama bittikten sonra hem Ar-Ge ofisinde hem de gerçek dünya şartlarında ürünün testleri yapılır. Son olarak doğrulama ve bakım aşaması zamana yayılan, ürün gerçek dünya şartlarında test edildikçe yazılımın ve donanımın iyileştirilmesini sağlayan aşamadır.

BÖLÜM 3. STANDARDİZASYON

3.1 Genel Mühendislik Standartları

3.1.1 ISO(International Standards Organization)

23 Şubat 1947’de İsviçre’nin Cenevre şehrinde kurulmuştur. Kuruluşunda 26 ülke kurucu üye olarak yer almıştır. Türkiye bu topluluğa 1955 yılında üye olmuştur. ISO’nun mevcuttaki üye sayısı 165’tir.

ISO’nun amacı;

- Uluslararası ticareti kolaylaştırmak
- Kalite ve güvenlik standartlarını tanımlamak
- Ulusal standartlar arasındaki farkları azaltmak
- Bilimsel ve teknik bilginin yayılmasına yardımcı olmak
- Teknolojik ilerlemelerle birlikte yeni ihtiyaçlara yönelik standartlar oluşturmak
- Standardizasyon ile ilgili diğer uluslararası kuruluşlarla iş birliği yaparak standardizasyon çalışmalarında bulunmaktır.



Şekil 3.1. ISO logo

3.1.1.1 ISO Standart oluşturma aşamaları

Öneri Aşaması(Proposal Stage)

- Yeni bir standardın gerekliliği ortaya konur.
- ISO üye ülkelerinden biri veya bir teknik komite, yeni bir standart önerisinde bulunur.
- Bu öneri diğer üyelerin oylamasına sunulur. Kabul edilirse süreç başlatılır

Hazırlık Aşaması(Preparatory Stage)

- Bir çalışma grubu(Working Group) kurulur.
- Uzmanlar taslak bir belge(Working Draft – WD) hazırlar
- Bu belge genelde ilk fikir taslağıdır ve detaylı teknik içerik barındırmaz.

Komite Aşaması(Committee Stage)

- Teknik komiteler veya alt komiteler, taslağı gözden geçirir.
- Taslak üzerinde görüşler bildirilir, değişiklikler yapılır.

Taslak Aşaması(Draft Stage)

- Üye ülkelere gönderilen taslak üzerinde resmi oylama yapılır.
- Draft International Standard(DIS) adıyla yayımlanır.
- Yeterli onay alınamazsa taslak tekrar düzenlenir.

Onay Aşaması(Approval Stage)

- Nihai taslak(Final Draft International Standard – FDIS) yayımlanır.
- Üye ülkeler son bir kez oylama yapar
- Oy çokluğu sağlanırsa standart onaylanmış olur.

Yayınlama aşaması(Publication Stage)

- Onaylanan standart, ISO tarafından resmi olarak yayımlanır.
- Tüm dünyaya duyurulur ve erişime açılır.
- Uygulamaya alınması için ülkeler kendi sistemlerinde tanımlar.

3.1.1.2 Başlıca ISO standartları

- ISO 9000 – Kalite Yönetim Sistemleri: Temel Kavramlar ve Terimler standardıdır. İlk yayın yılı 1987’dir. Güncel versiyonu ISO 9000:2015’dir. Kalite yönetim sistemlerine yönelik temel kavramları ve terimleri tanımlar. Kalite süreçlerinin ortak bir dil ile yürütülmesini sağlar ve ISO 9001 gibi uygulamalı standartların teorik temelini oluşturur.



Şekil 3.2. ISO 9000 logo

- ISO 14000 – Çevre Yönetim Standardı. İlk yayın yılı 1996’dır güncel versiyonu 2015 versiyonudur. Kuruluşların çevreye olan etkilerini sistematik bir şekilde yönetmelerini ve iyileştirmelerini amaçlar. Atık, enerji ve doğal kaynakların sorumlu kullanımı konusunda çerçeve sunar.
- ISO 26000 – Sosyal Sorumluluk Standartı. Yayın yılı 2010’dur. Kurumların topluma ve çevreye karşı sorumluluklarını etik ilkeler doğrultusunda yerine getirmelerine rehberlik eder. Yasalara, insan haklarına ve şeffaflığa saygıyı temel alır.

- ISO 31000 – Risk Yönetimi Standardı. İlk yayın yılı 2009, gün versiyonu 2018’dir. Her sektörde uygulanabilir genel bir risk yönetimi çerçevesi sunar. Kuruluşların belirsizliklerle başa çıkmasına ve karar alma süreçlerinde daha güvenli ilerlemesine katkı sağlar.
- ISO 37000 – Kurumsal Yönetim Standardı. Yayın yılı 2021’dir. Şirketlerin etik, şeffaf ve hesap verebilir bir şekilde yönetilmesini sağlayan bir kurumsal yönetim rehberidir. Sürdürülebilir kalkınma ve uzun vadeli değer üretimi hedeflenir.
- ISO 50000 – Enerji Yönetimi Standartları Serisi. İlk yayın yılı 2011’dir. Güncel uygulama standardı ISO 50001:2018’dir. Enerjinin verimli kullanımı, enerji maliyetlerinin azaltılması ve çevresel etkilerin düşürülmesi için rehberlik eder. ISO 50001 gibi uygulama standartlarının temelini oluşturur.
- ISO/IEC 27000 – Bilgi Güvenliği: Temel Kavramlar ve Tanımlar. İlk yayın yılı 2009’dur. Güncel versiyonları 2012 ve 2018’dir. Bilgi güvenliği yönetim sistemlerinin temel kavramlarını, terimlerini ve prensiplerini açıklar. ISO/IEC 27001 gibi standartların altyapısını destekler.
- ISO 45000 – İş Sağlığı ve Güvenliği Genel Çerçevesi. Yayın yılı 2018’dir. İş yerlerinde çalışanların sağlığının ve güvenliğinin korunması için oluşturulan uygulamalı sistemlerin genel prensiplerini tanımlar. Proaktif risk önleme yaklaşımını destekler.
- ISO 19011 – Yönetim Sistemleri Denetim Rehberi. İlk yayın yılı 2002’dir. Güncel versiyonu 2018’dir. Kalite, çevre, bilgi güvenliği gibi yönetim sistemlerinin iç ve dış denetimlerinin nasıl yapılması gerektiğini açıklayan bir denetim rehberidir. Denetçilerin görevlerini standart bir çerçevede yürütmesini sağlar.

3.1.2 IEC(International Electrotechnical Commission)

Elektrik, elektronik ve ilgili teknolojiler alanında uluslararası standartlar geliştiren bağımsız ve kar amacı gütmeyen bir kuruluştur. 1906 yılında Cenevre, İsviçre’de kurulmuştur. Türkiye’deki temsilcisi TSE(Türk Standartları Enstitüsü)’dür.

IEC amaçları;

- Elektrik ve elektronik mühendisliği alanında kullanılan ürün ve sistemler için küresel teknik standartlar geliştirmek
- Uyumlu, güvenli, verimli ve çevreye duyarlı teknolojilerin geliştirilmesine öncülük etmek.
- Ulusal teknik düzenlemelerin karşılıklı tanınmasını kolaylaştırmak.

3.1.2.1. IEC başlıca yapay zeka standartları

- ISO/IEC 22989:2022 – Yapay Zeka Kavramları ve Terminolojisi. Yayınlanma tarihi Temmuz 2022’dir. Bu standart, yapay zeka alanındaki temel kavramları ve terminoloji tanımlar. Yapay zeka ile ilgili diğer standartların geliştirilmesine ve farklı paydaşlar arasındaki iletişimin kolaylaştırılmasına yardımcı olur.
- ISO/IEC 23053:2022 – Makine Öğrenimi Kullanan Yapay Zeka Sistemleri İçin Çerçeve. Yayınlanma tarihi Temmuz 2022’dir. Bu standart, makine öğrenimi teknolojisini kullanan bir yapay zeka sisteminin bileşenlerini ve işlevlerini tanımlayan bir çerçeve sunar. Yapay zeka ekosistemindeki sistemlerin anlaşılması ve tasarımı için rehberlik sağlar.
- ISO/IEC 24028:2020 – Yapay Zekada Güvenilirlik Üzerine Genel Bakış. Yayınlanma tarihi Ekim 2020’dir. Bu teknik rapor, yapay zeka sistemlerinin güvenilirliğini sağlamak için mevcut yaklaşımları ve teknikleri inceler. Güvenilir yapay zeka sistemlerinin geliştirilmesi ve değerlendirilmesi için önemli hususları ele alır.
- ISO/IEC TR 24027:2021 – Yapay Zeka Sistemlerinde ve Yapay Zeka Destekli Karar Vermede Önyargı. Yayınlanma tarihi Kasım 2021’dir. Bu teknik rapor, yapay zeka sistemlerinde ve yapay zeka destekli karar verme süreçlerinde ortaya çıkabilecek önyargıları ele alır. Önyargının ölçülmesi, değerlendirilmesi ve azaltılması için yöntemler sunar.
- ISO/IEC TR 24029-1:2021 Sinir Ağlarının Dayanıklılığının Değerlendirilmesi. Yayınlanma tarihi Kasım 2021’dir. Bu teknik rapor, sinir ağlarının dayanıklılığını değerlendirmek için mevcut yöntemlerin bir genel bakışını sağlar. Yapay zeka sistemlerinin güvenilirliğini arttırmak için kullanılan teknikleri açıklar.

3.2 İşletmede Uygulanan Standartlar

Tablo 3.1 Başlıca Standartlar

Standart No	Standart
ISO 9000	Kalite Yönetimi – Temel Kavramlar ve terimler
ISO 9001	Kalite Yönetim Sistemi – Uygulanabilir Şartlar
ISO 14001	Çevre Yönetim Sistemi
ISO 45001	İş Sağlığı ve Güvenliği Yönetim Sistemi
ISO 31000	Risk Yönetimi Rehberi
ISO 19011	Yönetim Sistemleri Denetim Kılavuzu
ISO 26000	Sosyal Sorumluluk Rehberi
ISO 50001	Enerji Yönetim Sistemi
ISO/IEC 12207	Yazılım Yaşam Döngüsü Süreçleri
ISO/IEC 25010	Yazılım Kalite Modeli
ISO/IEC 29119	Yazılım Test Standartları
ISO/IEC 27001	Bilgi Güvenliği Yönetim Sistemi
ISO/IEC 27002	Bilgi Güvenliği Kontrolleri Uygulama Kuralları
ISO/IEC 27005	Bilgi Güvenliği Risk Yönetimi
ISO/IEC 2382	Bilgi Teknolojileri Terminolojisi
IEE 830	Yazılım Gereksinim Belirtimi Standardı

Tablo 3.1(Devam) Başlıca Standartlar

ISO/IEC 22989 (2022)	Yapay Zekâ – Kavramlar ve Terminoloji
ISO/IEC 23053 (2022)	Makine Öğrenmesi İş Akışı Çerçevesi
ISO/IEC TR 24027 (2021)	Yapay Zekada Önyargı ve Adalet Teknik Raporu
ISO/IEC TR 24028 (2020)	Yapay Zeka Sistemlerinin Güvenilirliği Üzerine Teknik Rapor
ISO/IEC TR 24029-1 (2021)	Yapay Zeka Güvenilirliğinin Değerlendirilmesi – Kısım 1
ISO/IEC DIS 42001 (2024)	Yapay Zeka Yönetim Sistemi Gereklikleri
OECD AI Principles (2019)	İnsan odaklı, adil, açıklanabilir yapay zeka
UNESCO AI Ethics (2021)	Kültürel ve etik değerlerle uyumlu yapay zeka ilkeleri
G20 AI Principles	Şeffaf, güvenli, denetlenebilir yapay zeka yönetimi
ISO/IEC 14496-10	Görüntü kodlama: H.264/MPEG-4 AVC standardı
IEEE 1857	Video kodlama, görüntü işleme sistemleri için standartlar
ISO/IEC 15938	Görsel içerik tanımlama (MPEG-7)
ISO/IEC TR 29138	Erişilebilirlik ve görsel bilgi sunumu teknikleri
EMVA 1288	Makine görüşü kameraları için performans ölçüm standardı
ISO 21500	Proje yönetimi rehberi
ISO/IEC 29110	Küçük yazılım kuruluşları için süreç yaşam döngüsü
IEEE 1063	Yazılım dokümantasyon standardı

3.3 İşletmedeki Kalite Yönetim Sistemi

AISOFT firması, mühendislik ve yazılım geliştirme süreçlerinde sürdürülebilir ve sistematik şekilde yönetebilmek amacıyla ISO 9001:2015 “Kalite Yönetim Sistemi” çerçevesinde çalışmaktadır. Firma bünyesinde kalite; yalnızca ürün veya hizmetin son aşamasında değil, tüm süreçlerin başından itibaren izlenebilirlik, tutarlılık ve sürekli iyileştirme felsefesiyle ele alınmaktadır.



Şekil 3.3. ISO 9001:2015 logo

Kalite yönetimi kapsamında; proje başlangıcında müşteri gereksinimleri net olarak belirlenmekte, yazılım geliştirme süreci boyunca gereksinim karşılaştırmaları yapılmakta, kod incelemeleri, test senaryoları ve doğrulama aşamaları titizlikle uygulanmaktadır. Ayrıca, iç denetimlerle kalite performansı izlenmekte ve gerekli durumlarda düzeltici faaliyetler devreye alınmaktadır.

Bilgisayarlı görü ve yapay zeka alanında faaliyet gösteren AISOFT, kaliteyi sadece teknik başarıyla sınırlı görmemekte; müşteri memnuniyeti, bilgi güvenliği, süreç yönetimi ve etik sorumluluk gibi çok boyutlu bir yaklaşımı benimsemektedir. Bu

doğrultuda, çalışanlara kalite kültürü kazandırılmakta ve kalite hedefleri kurum vizyonu ile entegre bir şekilde belirlenmektedir.

Firma ayrıca yazılım yaşam döngüsüne ait süreçlerini ISO/IEC 12207 standardına göre tanımlayarak kaliteyi yalnızca sonuçta değil, her adımda güvence altına almaktadır. Geliştirilen projelerde versiyon kontrolü, belge yönetimi, test doğrulamaları ve kullanıcı geri bildirim döngüleri kalite yönetiminin ayrılmaz birer parçasıdır.

3.3.1. Kalite yönetim süreci aşamaları

3.3.1.1. Planlama

- Projeye başlamadan önce müşteri ihtiyaçları ve teknik gereksinimler detaylı şekilde analiz edilir.
- Kalite hedefleri belirlenir(Örnek: teslim süresi sapması < %5, hata oranı < %1, maliyet vb.)
- Proje planına kalite kontrol adımları dahil edilir.
- Geliştirme süreci, yazılım yaşam döngüsüne göre(ISO/IEC 12207) yapılandırılır.

3.3.1.2. Uygulama

- Yazılım geliştirme süreçleri, çevik metodolojiler ve versiyon kontrol sistemleri(Git) ile yönetilir.
- Kodlama standartlarına uygunluk ve modüler yapı sağlanır.
- Ürün tasarımı, veri etiketleme ve model eğitim süreçlerinde kalite kriterleri tanımlanır.
- Girdi(veri setleri, kod, model parametreleri) ve çıktı(model doğruluğu, çıktı süresi) kontrol edilir.

3.3.1.3. Kontrol

- Yapay zeka modelleri için performans değerlendirme metrikleri(mAP, F1-Score) raporlanır.
- İç denetimler, kalite hedeflerine uygunluğu izlemek için periyodik olarak gerçekleştirilir.
- Müşteriden düzenli geri bildirim alınarak analiz edilir ve memnuniyet ölçülür.

3.3.1.4. Önlem ve İyileştirme

- Hata tespit edilen alanlarda düzeltici ve önleyici faaliyetler planlanır.
- Tespit edilen yazılım hataları, eksik gereksinimler ve yetersiz model çıktıları döngüsel olarak iyileştirilir.
- Süreç performansı analiz edilerek kalite hedefleri güncellenir.

BÖLÜM 4. İŞYERİ UYGULAMALARI

4.1. Kaynak Kodu Güvenliği

Kaynak kodlarının korunmasına dair çalışma yapıldı. İşyerinde bazı projeler Docker konteynerleri aracılığıyla ayağa kaldırılmakta/çalıştırılmaktadır. Bunun sebebi yazılımların her platformda aynı şekilde çalışmasının garanti edilmesi ihtiyacıdır.

4.1.1. Docker

Docker, yazılım uygulamalarını taşınabilir, hafif ve izole şekilde çalıştırmak için kullanılan bir platformdur. Uygulamalar ve tüm bağımlılıklarıyla birlikte “konteyner” adı verilen yapılarda paketlenmesini sağlar. Böylece bir uygulama, geliştiricinin bilgisayarında nasıl çalışıyorsa, sunucuda da aynı şekilde çalışır.



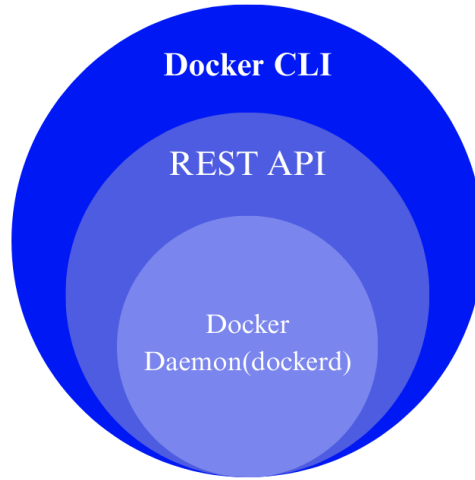
Şekil 4.1. Docker logo

4.1.1.1. Docker çalışma mantığı

Docker’ın çalışma mantığı, işletim sistemi seviyesinde sanallaştırma kullanarak uygulamaları izole konteynerlerde çalıştırmaktır. Konteynerler, ana işletim sistemi çekirdeğini paylaşır ama kendi dosya sistemine, ağ yapılandırmasına ve işlem alanına sahiptir. Bu sayede bağımlılık çakışmaları gibi sorunlarla karşılaşma ihtimali ortadan kaldırılır.

Docker Engine: Docker’ın temel bileşenidir. 3 parçadan oluşur.

- Docker Daemon(dockerd): Arka planda çalışır, konteynerleri yönetir.
- Docker CLI:(“docker” komutu) kullanıcının komut yazdığı arayüzdür
- REST API: Diğer uygulamaların Docker ile konuşmasını sağlar



Şekil 4.2. Docker engine parçaları

Docker Image(Docker İmajı): Bir uygulamanın çalışması için gereken dosyalar, bağımlılıklar ve ayarlar tek bir dosya halinde paketlenir.

Docker Container(Docker Konteyner): İmajdan oluşturulan, çalışan canlı bir uygulamadır. İmaj çalıştırıldığında Docker izole bir süreç başlatır ve bu süreç bir konteyner olur. Her konteyner, kendi dosya sistemine ve ağ arabirimine sahiptir ama çekirdek(kernel) işletim sistemiyle ortaktır. Bu sayede klasik sanal makinelerden çok daha hafiftir

4.1.1.2. Docker'ın temel özellikleri

- Konteyner(Container): Uygulamanın çalışması için gereken her şeyi(kod, kütüphaneler, ayarlar) içinde barındırır.
- İzolasyon: Her Konteyner diğerlerinden bağımsız çalışır. Bir konteynerdeki hata diğerini etkilemez.
- Taşınılabilirlik: Geliştirilen uygulama “her yerde çalışır” felsefesine uygundur(Windows, Linux, Mac, sunucu vb.)
- Hafiflik: Sanal makine mantığından farklı olarak tam bir işletim sistemi taşımaz, daha az kaynak kullanılır.

4.1.3. Problem, sunulan çözümler ve karşılaşılan zorluklar

İşyerinde Docker kullanılmakta, yazılımlar Docker konteynerlerinde ayağa kaldırılmaktadır. Docker konteynerlerinde kaynak kodlarının korunmasına dair çalışma yapılmıştır.

Docker’da yazılım süreçlerinin standartlaştırılması, taşınabilir hale getirmesinin yanında güvenlik açıklarına da sahiptir. Bunun örneklerinden birisi yönetici şifresine sahip birisinin imaj ve konteynerdeki kaynak kodlarına erişebilmesidir. Host(ana) makinanın şifresinin verilmesinin gerekli olduğu durumlarda kaynak kodunun korunması önemli bir sorun olarak ortaya çıkmaktadır. Bu durumun çözülmesi için araştırmalar yapılmıştır.

4.1.3.1. Hukuki yöntem

Kaynak kodunun korunmasında başta gelen uygulamalardan biri olarak hukuki yöntemler önerildi. Sistemlerin kırılmaz, önlemlerin atlatılamaz olmaması dolayısıyla projede işyerinin ticari ürününü uygun sözleşmelerle koruması gerekir.

4.1.3.2. Docker imajı içinde kaynak kod şifreleme

Kodlar AES, RSA gibi yöntemlerle şifrenip imaj içinde saklanır. Uygulama çalışırken RAM üzerinde çözülür. Kod disk üzerinde açık halde tutulmaz. Dezavantaj olarak şifre çözme işlemi RAM üzerinde yapılacağından dolayı çalışma performansı etkilenebilir. Bunun yanı sıra bu yöntemde güvenli anahtar yönetimini gerekir. Şifreleme/çözme entegrasyonunu sağlamak karmaşıktır. Zayıf nokta olarak sudo yetkisine sahip kullanıcı, RAM dump(RAM’in anlık görüntüsünün dosya olarak alınması) yaparak veya debugging araçları ile çözülmüş koda ulaşabilir.

4.1.3.3. Docker build sürecinde kaynak kodun sadece geçici olarak kullanılması

Multi-stage build yapısıyla kod yalnızca ilk aşamada bulunur. Final imajda sadece derlenmiş çıktılar yer alır; kaynak dosyalar dahil edilmez. Avantaj olarak final imaj dağıtıldığında kaynak kod dışı sızmaz. Otomasyonla her build’de kod derlenir, dışı aktarılmaz. Dezavantaj olarak Python, Javascript gibi script tabanlı diller bu avantajdan kısıtlı şekilde faydalanır, çünkü yorumlayıcı ortamda kaynak kod gerekebilir. İşyerinde projeler genel olarak Python üzerinden yürüdüğü için bu yöntem zayıf yöntemlerden biridir. Zorluk olarak Dockerfile yapısının iyi kurgulanması gerekir. Katmanların yönetimi, build cache’in doğru temizlenmesi önemlidir. Build cache, Docker’ın imaj oluşturma sürecini hızlandırmak ve tekrar eden işlemleri önlemek için kullandığı ara bellektir. İlgili dosyalar değişmediyse yeniden kullanır bu da build sürecini çok daha hızlı hale getirir. Bu yöntemin zayıf noktası şudur: Sudo erişimi olan birisi, docker history komutuyla image geçmiş katmanlarını inceleyerek kaynak kodun kullanıldığı aşamaya ulaşabilir.

4.1.3.4. Read-only(salt okunur) volume ile kodun değiştirilememesi

Bu yöntem kaynak kodunun okunmasının engellenmesinden ziyade kodun değiştirilmesi, silinmesi gibi konulara engel olur. Kaynak kod konteyner içerisinde read-only olarak mount edilir. Böylece uygulama veya dış kullanıcı kodu değiştiremez. Dezavantajı kodun hala okunabilir olmasıdır ancak sistemin ayakta/çalışır halde kalmasına yardımcı olur. Zayıf nokta olarak sudo yetkisi olan birisi volume’u yeniden mount edip read-write hale getirebilir veya dosyaları dışarıya kopyalayabilir.

4.1.3.5. Derlenmiş binary dosyalarının kullanımı

Kaynak kod, çalıştırılabilir hale getirilir. Docker image’ına yalnızca bu binary eklenir, kaynak dosyalar tamamen hariç tutulur. Binary dosyalar yorumlanabilir kodlara göre tersine mühendislik açısından daha dirençlidir. Kod doğrudan okunamaz. Dezavantaj olarak debug işlemleri zordur. Yine de binary analiz araçları ile sınırlı bilgi alınabilir. Bunun yanı sıra farklı mimariler için yeniden derleme gerektirebilir. Sudo yetkisine sahip kullanıcı binary’i dışarı çıkararak Ghidra, IDA Pro, Radare2 gibi araçlarla dosyayı analiz edebilir.

```
Staj raporu > Dockerfile
1 #Kodun Derlenmesi
2 FROM python:3.11-slim AS builder
3
4 WORKDIR /app
5 COPY requirements.txt .
6 RUN pip install --no-cache-dir -r requirements.txt
7 COPY . .
8
9 # PyInstaller ile tek bir dosya haline getirilmesi
10 RUN pip install pyinstaller && \
11     pyinstaller --onefile app.py
12
13 # 2. Aşama: Sadece çalıştırılabilir binary dosyasının taşındığı final
14 FROM python:3.11-slim AS final
15
16 WORKDIR /app
17
18 # Final imaja sadece exe dosya alınıyor
19 COPY --from=builder /app/dist/app /app/app
20
21 # Giriş noktası
22 CMD ["./app"]
```

Şekil 4.3. Multi-Stage build ve binarye çevirme

4.1.3.6. Distroless veya scratch image kullanımı

Bu image türleri işletim sistemi yardımcı araçlarını içermez. Uygulama dışında neredeyse hiçbir komut çalıştırılmaz. Avantaj olarak “docker exec”, “bash”, “sh” gibi yöntemlerle konteynere girilse bile içeride komut çalıştırmak mümkün olmaz. Dezavantaj olarak debug işlemleri zorlaşır. Hata ayıklamak için ek yöntemler gerekir. İşyerinde Ubuntu imajı kullanma zorunluluğu önerilen bu yöntemi geçersiz kılmaktadır.

4.1.3.7. Kod obfuscation(karmaşılaştırma) ile anlaşılabilir hale getirme

Kaynak kodun okunmasını ve analiz edilmesini zorlaştırmak için obfuscation araçları kullanılır. Fonksiyon isimleri, değişkenler ve mantık yapıları karmaşılaştırılır. Kod çözümlemesi ve tersine mühendislik ciddi manada zorlaşır. Dağıtılan kod teknik olarak koruma altına alınır. Koddaki her değişiklikte obfuscation işleminin tekrarlanması gerekir. Dezavantaj olarak performans etkilenebilir ve kod okunurluğu tamamen kaybolur. Sudo kullanıcısı obfuscate edilmiş kodu dışa aktarıp analiz edebilir, ayrıca obfuscation'ı geri çözen araçlar mevcuttur.

```
1
2 def fonksiyon():
3     print("fonksiyon.")
4
5 def ana_program():
6     fonksiyon()
7
8 if __name__ == "__main__":
9     ana_program()
10
11
12 #karmaşılaştırılmış kod.
13 def BfKLE38sLZoKuhc():
14     print("fonksiyon.")
15
16 def ujeX20LkjY():
17     BfKLE38sLZoKuhc()
18
19 if __name__ == "__main__":
20     ujeX20LkjY()
21
```

Şekil 4.4. Kod karmaşılaştırma

4.1.3.8. Bellekte çalışan kodun diskte iz bırakmadan yürütülmesi

Kod dosyaları diske yazılmadan, ağ üzerinden belleğe alınarak doğrudan çalıştırılır. Genellikle exec() gibi dinamik çalışma teknikleriyle birlikte kullanılır. Dosya sistemi üzerinde hiçbir iz bırakmaz, bu da statik analiz araçlarının başarısını ciddi oranda azaltır. Dezavantaj olarak uygulama çalışması sırasında ağ bağlantısına ihtiyaç duyabilir ve bellekte şifrelenmemiş halde kod tutulması risk oluşturabilir.

Ağ güvenliği, zamanlama, bellek yönetimi ve geçici veri yaşam döngüsünün dikkatli kurgulanması gerekliliği zorluktur. Zayıf nokta olarak Sudo kullanıcısı gcore, strace, procfs veya doğrudan fiziksel bellek analizi ile bellekteki kodu elde edebilir.

4.1.3.9. Lisans sunucusu doğrulaması ile yetkisiz çalışmasının engellenmesi

Uygulama başlatılırken lisans sunucusuna bağlanarak cihaz, kullanıcı, tarih gibi parametrelere göre doğrulama yapılır. Geçerli lisans yoksa uygulama kapanır. Yazılım sadece lisanslı kullanıcılar tarafından kullanılabilir. Lisansız kopyaların çalışması engellenir. Dezavantaj olarak lisans sunucusu çökerse uygulama çalışmaz.

Çevrimdışı kullanım için ek yapılandırma gerekir. Lisans doğrulama altyapısı kurmak, yönetmek ve güvenliğini sağlamak ciddi teknik bilgi ister.

SUDO kullanıcı, ağ doğrulamasını geçersiz kılan sahte bir sunucu kurabilir veya doğrulama fonksiyonunu bypass eden patch'ler uygulayabilir.

4.1.3.10. AppArmor/SELinux/seccomp ile sistem erişimini sınırlama

Bu güvenlik mekanizmaları konteynerin çalıştığı çekirdek düzeyinde, uygulamanın hangi sistem çağrılarını yapabileceğini veya hangi dosyalara erişebileceğini kontrol eder. Ağ, dosya sistemi, belleğe özel alanlar gibi sistem kaynaklarına erişim sınırlandırılabilir. Dezavantaj olarak yanlış yapılandırma uygulamanın çalışmasını engelleyebilir. Hata ayıklamak zorlaşır. Bu yöntemde politika dosyaları oluşturmak ve test etmek zaman alır. Geliştirici deneyimi gerektirir. Sudo yetkilerine sahip kullanıcı bu profilleri geçici olarak kaldırabilir veya sistemin koruma modlarını devre dışı bırakabilir.

4.1.3.11. Docker secrets ile hassas bilgilerin korunması

Docker swarm ortamında, uygulamaya ait şifreler, tokenler veya API anahtarları gibi gizli bilgiler özel bir yöntemle şifreli ve erişim kontrollü olarak iletilir. Ortam değişkenlerinden daha güvenli bir çözüm sunar. Diskte şifreli halde tutulduğu için erişim sınırlıdır. Dezavantaj olarak sadece Docker Swarm modunda kullanılabilir, standart konteynerlerde bu yapı çalışmaz. CI/CD pipeline'ları ile uyumlu hale getirmek ve geliştirme ortamında test edebilmek için ek yapılandırmalar gerekir. Sudo yetkisi olan bir kullanıcı secrets'in mount edildiği klasöre ulaşarak veriye erişebilir.

4.1.3.12. Shell ve debug araçlarının konteynerden kaldırılması

Konteyner içinde bash, sh, python gibi yorumlayıcılar ve debug araçları kaldırılır. Böylece dışarıdan erişimle analiz yapılması zorlaşır. “docker exec” gibi komutlarla konteyner içine girilse bile içeride işlem yapılamaz. Kod doğrudan analiz edilemez. Buna karşılık hataların çözümü ve uygulama içi müdahale zorlaşır. Konteyner dışı loglama zorunlu hale gelir. Gerekli tüm işlem ve log mekanizmalarının baştan yapılandırılma zorunluluğu vardır. Sudo yetkisi olan kullanıcı yeni Shell binarylerini yükleyerek bu korumayı aşabilir.

4.1.3.13. Anti-Tamper mekanizmaları

Kod dosyalarının hasleri ya da imzaları çalıştırma sırasında kontrol edilir. Herhangi bir dosyada değişiklik tespit edilirse uygulama kendini kapatır. Kodun değiştirilmesi veya kurgulanması anında tespit edilir. Otomatik koruma mekanizması sağlar. Yanlış pozitif tespitlerde uygulama gereksiz yere kapanabilir. Bu durum kullanıcı deneyimini olumsuz etkiler. İmza oluşturma, doğrulama mantığı ve hata yönetimini tasarlamak

zorludur. Sudo kullanıcısı uygulamanın kontrol algoritmasını doğrudan değiştirerek bypass edebilir.

4.1.3.14. TPM/SGX gibi donanım tabanlı güvenlik sistemleriyle entegrasyon

Uygulama, belirli donanımlarla özel çalışacak şekilde şifrelenir. TPM veya SGX ile çalıştırma yetkisi doğrulanır. Bu sayede kod sadece belirli cihazlarda çalışacak şekilde koruma altına alınabilir. Dışa sızan kod çalıştırılmaz. Buna karşılık donanım uyumluluğu ve üreticiye bağımlılık gibi kısıtlamalar vardır. Donanım seviyesinde destek gerektirdiği için test ve dağıtım süreçleri karmaşıklaşır. Sudo kullanıcısı BIOS/firmware ayarlarını değiştirerek bu güvenlik kontrollerini devre dışı bırakabilir.

4.1.3.15 Kodun uzaktan API'den alınarak RAM'de çalıştırılması

Kod veya model ağırlıkları konteyner başlatıldığında bir API'dan alınır, bellekte çözülür ve geçici olarak çalıştırılır. Diskte hiçbir zaman saklanmaz. Kodun konteyner içinde fiziksel varlığı bulunmaz. Kod yalnızca çalışma anında ve geçici bellekte yer alır. Buna karşılık internet bağlantısı olmadan uygulama çalışmaz. API güvenliği de ayrı bir meseledir. API erişimi, şifreleme, anahtar yönetimi ve bellek temizliği özel dikkat ister. Sudo yetkili kullanıcılar API trafiğini analiz ederek kodu elde edebilir veya RAM'de çalıştırılan kodu çıkarabilir.

4.1.3.16. Özel EntryPoint script ile ortam doğrulama

Uygulama başlatılmadan önce IP adresi, cihaz kimliği, zaman damgası gibi bilgileri kontrol eden scriptler çalıştırılır. Koşullar sağlanmazsa uygulama başlatılmaz. Bu sayede uygulama yalnızca belirli ortam veya koşullar altında çalışır. Sızdırılsa dahi her yerde çalışmaz. Buna karşılık bu kontroller atlatılabilir, sabit değerler verilerek kandırılabilir. Bu yöntemde güvenilir kontrol mekanizmaları ve zaman senkronizasyonu gerekir. Sudo kullanıcısı ortam değişkenlerini taklit edebilir veya scriptin kontrolünü değiştirebilir.

4.1.3.17. Özel ve şifreli Docker registry kullanımı

Docker imajları özel ve şifreli registrylerde saklanır. Yetkisiz kullanıcıların imajlara erişimi engellenir. Dağıtım süreci daha güvenli bir hal alır. Buna karşılık registry kurulumu, bakımı ve güvenlik duvarı yapılandırması ek yük getirir. Bir diğer taraftan CI/CD süreçlerine bu yapıların entegre edilmesi karmaşık olabilir. Sudo kullanıcısı imajı export ederek registry dışında paylaşabilir.

4.1.3.18. Şifrelenmiş dosya sistemi(LUKS, eCryptFS) kullanımı

Docker'ın içeriği, şifreli bir dosya sistemine kurulur. Veriler disk üzerinde yalnızca şifreli biçimde tutulur. Fiziksel erişim durumunda veriye ulaşılamaz. Diskin tamamı

koruma altına alınmış olur. Buna karşılık disk performansı düşebilir ve erişim gecikmeleri yaşanabilir. Bir diğer taraftan anahtar yönetimi ve sistem başlatma senaryolarında özel entegrasyon gerekir. Bu yöntemde sudo kullanıcısı root yetkisiyle mount işlemini gerçekleştirebilir ve diski okunabilir hale getirebilir.

4.1.3.19. Docker layer temizliği ve minimal image üretimi

Docker build sırasında geçici dosyaların bulunduğu katmanlar silinir. Final image yalnızca çalışması için gerekli dosyaları içerir. Docker history çıktısında kaynak kod görünmez, image boyutu düşer, güvenlik artar. Eğer layerlar yanlış yapılandırılırsa koruma etkisiz kalır. Bu yöntemde Docker'ın katman mantığını iyi anlamak ve doğru sırayla build yapmak gerekir. Sudo kullanıcısı önceki intermediate image'lara erişerek dosyaları kurtarabilir.

4.1.3.20. Runtime ortamda ağ bağlantısı olmadan çalışacak şekilde tasarlama

Uygulama çalışırken dış dünyayla hiçbir ağ bağlantısı kurmaz. Ağ erişimi sadece kurulumda yapılır. Bu sayede kodun dışarıya veri sızdırması engellenir. Uygulama içi veri güvenliği artar. Buna karşılık güncellemeler, hata raporlamaları ve izleme sistemleri çalışmaz. Tüm veri içe kapalı olarak yapılandırılmalı, dışarıdan gelen loglama ihtiyacına yönelik özel sistemler kurulmalıdır. Sudo kullanıcısı geçici olarak ağ erişimini yeniden açarak uygulama davranışlarını değiştirebilir.

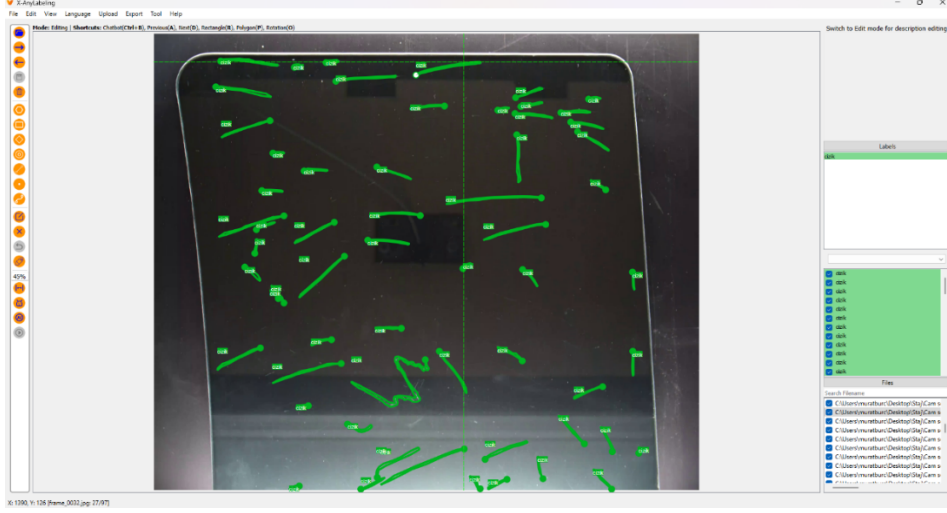
4.2. Veri Etiketleme

Veri etiketleme, ham veriye(görüntü, metin, ses, video, vb.) anlamlı açıklamalar veya etiketler eklenmesi sürecidir. Bu etiketler, yapay zeka ve makine öğrenmesi algoritmalarının veriyi öğrenebilmesini ve sınıflandırma, tespit, tahmin gibi görevleri yerine getirebilmesini sağlar.

Kaliteli veri yapay zeka model eğitiminde olmazsa olmaz konulardan birisidir. Bu durum doğrudan model performansını etkiler. Günümüzde veri etiketleme işini devretmenin yolları olsa da çoğu zaman etiketlemenin bilen kişiler tarafından yapılması elzemdir. Buna örnek olarak medikal taraftaki etiketlemeler örnek verilebileceği gibi bulunduğum işyerindeki etiketleme görevi de örnek verilebilir. Model performansının yüksek olması için etiketlemenin hangi sınırlar çerçevesinde yapılacağı önem kazanmaktadır.

Etiketli veriler, modelin doğruluğunu doğrudan etkiler. Kalitesiz etiketleme hatalı öğrenmeye sebep olur. Veri etiketlemesi denetimli öğrenme(Supervised Learning) algoritmalarının temelidir, bu öğrenme tipi bol miktarda etiketlenmiş veriye ihtiyaç

duyar. İyi etiketlenmiş veri, modelin gerçek dünya şartlarında daha başarılı sonuç vermesini sağlar.



Şekil 4.5. Etiketleme örneği

4.2.1. Veri etiketleme yöntemleri

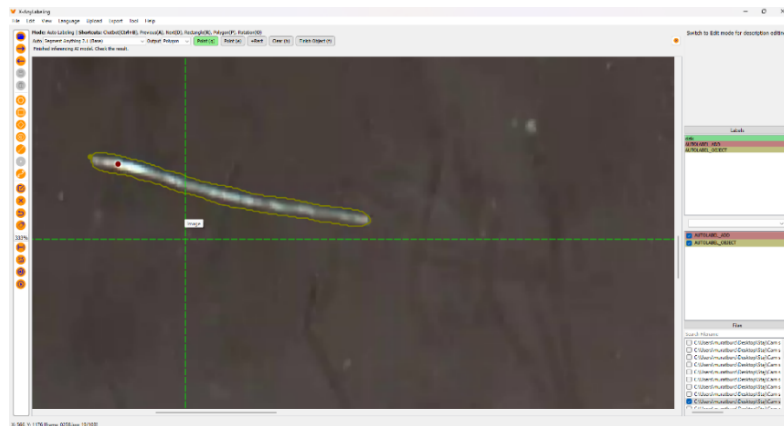
Veri etiketlemesinde çeşitli yöntemler bulunmaktadır. Bu yöntemlerde temel değişken zaman maliyetidir. Etiketlenecek verinin tipi belli bir yetkinlik seviyesini de gerektirebilir.

4.2.1.1. Manuel etiketleme

İnsanlar tarafından yapılır. Yüksek doğruluk sağlar ancak en zaman alıcı yöntemdir. LabelImg, CVAT, Label Studio, X-AnyLabeling gibi uygulamalar kullanılarak yapılabilir.

4.2.1.2. Yarı otomatik etiketleme

Bu yöntemde önce model etiketleme yapar, ardından bu etiketleri bir insan gözden geçirir. Manuel etiketlemeye kıyasla daha verimli olsa da en hızlı yöntem değildir. Özellikle büyük veri setlerinde tercih edilir.



Şekil 4.6. Yapay zeka destekli etiketleme örneği

4.2.1.3. Otomatik etiketleme

Tamamen model temelli etiketlemedir. Genelde önceden eğitilmiş bir model(YOLO vb.) kullanılarak görüntüler etiketlenir. Doğrudan büyük veri setleriyle eğitilmiş modeller kullanılabileceği gibi bu modellere özel data seti ile transfer öğrenmesi yapılarak otomatik öğrenme süreci hassaslaştırılabilir. Etiketler daha sonra insan doğrulaması ile kontrol edilebilir.

X-AnyLabeling, Roboflow Annotate, CVAT, Label Studio, MakeSense.ai otomatik etiketleme özelliğine sahip bazı etiketleme araçlarıdır. Biz segmentasyon etiketlemesi yaparken X-AnyLabeling uygulamasında Segment Anything Base 2.1 modelini olabildiğince kullandık ancak çoğu durumda daha hassas etiketleme için manuel ve daha özenli şekilde etiketlemeye özen gösterdik. Çalıştığımız veri setinde amacımız en küçük çizikleri dahi tespit etmek olduğundan ötürü Segment Anything Base 2.1 modeli çok bariz çiziklerin dışında başarılı sonuç vermekten uzak bir performans gösterdi. X-AnyLabeling uygulaması kendi eğittimiz modelleri kullanarak eğitim imkanı vermektedir. Bu yolu kullanmak için çalışmalarımız devam etmektedir. Otomatik etiketleme araçlarında kullanacağımız modelin bizim istediğimiz kalite ve hassaslıkta otomatik etiketleme yapabilmesi için bol miktarda manuel etiketlemesi gerekmesinin yanı sıra YOLO Segmentation, U-Net gibi farklı modellerin aynı eğitim setinde test edilerek en doğru modelin bulunması gerekmektedir.

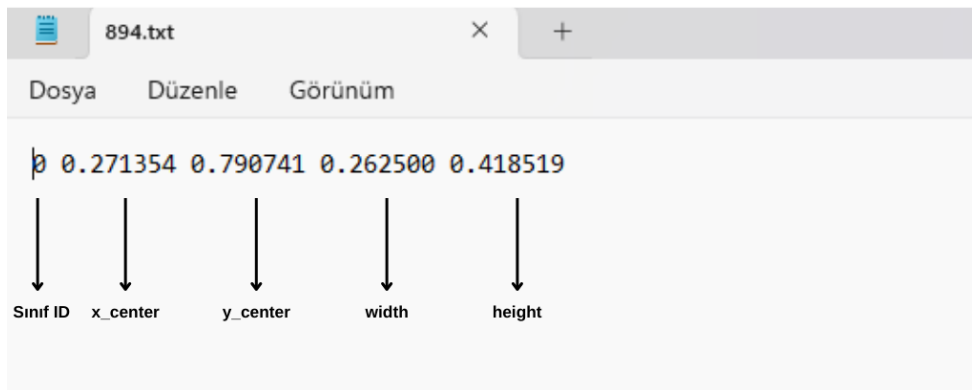
4.2.1.4. Aktif öğrenme

Bu yöntemde modelin en çok zorlandığı veriler seçilir ve sadece bu veriler etiketlenir. Etiketleme maliyetini azaltan bir yöntemdir.

4.2.2. Yaygın veri etiketleme formatları

4.2.2.1. YOLO TXT

.txt dosya formatıdır. Her satırda “class_id x_center y_center width height” değerleri bulunur.



Şekil 4.7. YOLO txt dosyası örneği

YOLO formatı, nesne tespiti(object detection) için kullanılan, sade ve hızlı işlenebilir bir etiketleme formatıdır. Genelde .txt uzantılı dosyalarla kullanılır ve her görüntü için bir tane .txt dosyası bulunur.

- Sınıf ID: Nesnenin ID'sini belirtir.
- x_center: Sınırlayıcı kutunun(bounding box) merkezinin yatay eksenindeki konumudur. 0-1 arasında normalleştirilmiştir. Örnek olarak görüntü 640 piksel genişliğindeyse $x_center_pixel = 0,271354 \times 640 = 173,67$ piksel
- y_center: Sınırlayıcı kutunun(bounding box) merkezinin dikey eksenindeki konumudur. 0-1 arası normalleştirilmiştir. Örnek olarak görüntü 480 piksel yüksekliğindeyse $y_center_pixel = 0,790741 \times 480 = 379,55$ px
- width: Kutunun genişliğidir. 0-1 arası normalleştirilmiştir. Görüntüye oranla kutunun yatay uzunluğudur. 640 piksel genişlikte bir görüntüde $0,262500 \times 640 = 168$ piksel genişliğe sahiptir.
- Height: Kutunun yüksekliğidir. 0-1 arası normalleştirilmiştir. Görüntüye oranla kutunun dikey uzunluğudur. Örnek olarak 480 piksel yüksekliğinde bir görüntüde $0,418519 \times 480 = 200,89$

4.2.2.2. Pascal VOC

Bu format .xml formatıdır. Sınıf adı ve sınırlayıcı kutu(bounding box) bilgisi içerir. Pascal VOC, bilgisayarla görme projelerinde yaygın olarak kullanılan bir nesne tespiti veri kümesi etiketleme formatıdır. Dosya formatı olarak .xml'i kullanır ve her bir görüntü dosyası için aynı adda .xml dosyası bulunur.

```
<?xml version="1.0" encoding="UTF-8" standalone="no" ?>
<annotation>
  <folder>frames_custom_60_30</folder>
  <filename>2.jpg</filename>
  <path>C:\bitirme\frames_custom_60_30\2.jpg</path>
  <source>
    <database>Unknown</database>
  </source>
  <size>
    <width>1920</width>
    <height>1080</height>
    <depth>3</depth>
  </size>
  <segmented>0</segmented>
  <object>
    <name>kucuk_piramit</name>
    <pose>Unspecified</pose>
    <truncated>0</truncated>
    <difficult>0</difficult>
    <bndbox>
      <xmin>317</xmin>
      <ymin>462</ymin>
      <xmax>437</xmax>
      <ymax>563</ymax>
    </bndbox>
  </object>
</annotation>
```

dosya yolu

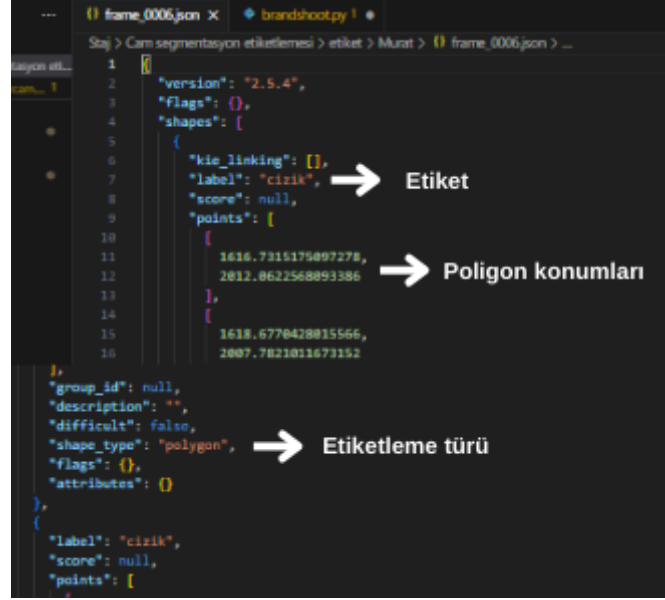
Genişlik ve yükseklik

Sınırlayıcı kutu koordinatları

Şekil 4.8. Pascal VOC XML örneği

4.2.2.3. LabelMe JSON

Bir görüntüye ait nesne konumlarını, sınıflandırmalarını ve ilgili meta verileri tutmak için JSON dosyaları kullanır.



Şekil 4.9. LabelMe JSON formatı örneği

- label: Etiket adı
- points: Poligon noktaları
- shape_type: “polygon” poligon ile çokgen etiketleme yapıldığını ifade eder

4.3. HIKROBOT Kamera Python Entegrasyonu

Staj sürecinde görüntü işleme uygulamaları kapsamında HIKROBOT MV-CS200-10GM ve HIKROBOT MV-CS060-10GM kameraların Python dili ile entegrasyonunu ve MV-CS200-10GM modelinin buton ile tetiklenmesini gerçekleştirdim. Bu kameralar GigE(Gigabit Ethernet) arayüzü ile bilgisayara bağlanmakta ve PoE Switch’den 48V ile güç almaktadır. Kamera kontrolü, üretici tarafından sağlanan Machine Vision SDK(MVS SDK) aracılığıyla yapıldı.

MVS SDK’nın Python arabirimi kullanılarak cihaz tespiti, bağlantı kurulması, çekim parametrelerinin ayarlanması ve görüntü yakalama işlemleri kodlanmıştır.

Python ile kodlanan uygulamada MVS SDK içerisindeki MvCameraControl_class.py modülü temel alınarak kamera nesnesi yönetilmiştir. Bu modül aracılığıyla kameranın exposure ayarı yapılmış, trigger ve freerun modları arasında seçim yapma imkanı

sağlanmıştır. Bunun yanı sıra trigger modda fiziksel buton ile tetikleme sağlanmış, butona kısa veya uzun basma tercihinə göre farklı sonuçlar elde edilmiştir.

4.3.1. HIKROBOT markası ve kullanılan kameralar

HIKRobot, 2016 yılında kurulmuş ve merkezi Çin'in Hangzhou şehrinde bulunan, makine görüşü(machine vision), endüstriyel otomasyon ve yapay zeka alanlarında ürün ve çözümler geliştiren bir teknoloji firmasıdır. Firma, Hikvision çatısı altında faaliyet göstermektedir ve özellikle endüstriyel kameralar, akıllı kontrol kartları, robotik sistemler ve görüntü işleme yazılımları ile tanınmaktadır.

Tablo 4.1 – HIKROBOT Kamera özellikleri

Özellik	MV-CS200-10GM	MV-CS060-10GM
Sensör	Sony IMX 183 CMOS	Sony IMX178 CMOS
Çözünürlük	5472 x 3648	3070 x 2048
Sensör Boyutu	1"	1/1.8"
Maks. Kare Hızı	5.9 FPS @ Tam çözünürlük	19.1 FPS @ Tam çözünürlük
Pozlama Süresi(exposure time)	46 μ s – 2.5 s	25 μ s – 2.5 s
Güç Girişi	12-24 VDC, PoE destekli	12-24 VDC, PoE destekli
Veri Arayüzü	Gigabit Ethernet	Gigabit Ethernet
I/O Portaları	6-pin Hirose	6-PIN Hirose
İşletim sistemi	Windows, Linux, macOS	Windows, Linux, macOS

Her iki kamera da endüstriyel görüntü işleme uygulamaları için tasarlanmış olup, GigE Vision standardını desteklemektedir. Bu sayede, yüksek çözünürlüklü görüntülerin düşük gecikme ile iletilmesi mümkün olmaktadır.,



Şekil 4.10 HIKROBOT MVS kamera ve etiket bilgileri

4.3.1.1. GigE vision

GigE Vision, makine görüşü sistemlerinde kullanılan bir kamera iletişim standartıdır. 2006 yılında AIA(Automated Imaging Association) tarafından geliştirilmiştir. Bu Standart, endüstriyel kameraların gigabit ethernet altyapısı üzerinden bilgisayarlarla veri alışverişi yapmasını sağlar. GigE Vision, hem fiziksel bağlantı hem de yazılım protokolü içeren bir çerçevedir.

4.3.1.2. 6-Pin hirose I/O konnektörü

Endüstriyel kameraların çoğunda kameraya harici bağlantılar yapılmasını sağlayan 6-pin Hirose I/O konnektörü bulunur. Bu konnektör, kameranın harici sistemlerle etkileşim kurmasını sağlar. Bu girişle harici tetikleme, tetikleme çıkışı, harici güç girişi gibi işlevler gerçekleştirilebilir.

- Pin 1 - DC 12V Giriş: PoE alternatif güç girişidir
- Pin 2 - GND: Toprak hattı
- Pin 3 - Opto-Isolated Input + (IN+): Harici tetikleme sinyali için pozitif uç
- Pin 4 - Opto-Isolated Input - (IN-): Harici tetikleme sinyali için negatif uç
- Pin 5 - Opto-Isolated Output + (OUT+) Kamerada işlem sonrası sinyal göndermek için kullanılabilir
- Pin 6 - Opto-Isolated Output - (OUT-) Output devresinin negatif hattı



Şekil 4.11. HIKROBOT Kamera 6-Pin hirose ve PoE girişi

4.3.2. Machine vision software development kit

MVS SDK, HIKROBOT tarafından geliştirilen bir makine görüş yazılım geliştirme kitidir. Bu yazılım paketi, HIKROBOT markalı endüstriyel kameraların bilgisayar sistemleri ile haberleşmesini ve kamera üzerinde tam kontrol kurulmasını sağlar. MVS SDK, kullanıcıların HIKROBOT ile kendi özel görüntü işleme yazılımlarını geliştirmesine imkan tanır.

4.3.3. Kamera ile iletişimde kullanılan kod

Bu kod, MVS SDK tarafından sunulan MvCameraControl_class.py modülünü kullanan hik_camera.py modülü çağrılarak yazıldı. MvCameraControl_class.py C/C++ ile yazılmış olan MVS SDK'sının .dll dosyalarını kullanarak kameranın kontrolünü Python üzerinden gerçekleştirmemizi sağlar.

```
import cv2
import time
import os
import traceback
from hik_camera.hik_camera import HikCamera

#başka kodlar içerisine monte edilebilmesi için class şeklinde tanımlandı
class HikFlexibleCamera:
    def __init__(self):
        self.cam = None
        self.ip = None
        self.mode = None # "trigger" veya "freerun" veya "usb"

#alttaki fonksiyon trigger modun fonksiyonudur.
def setup_trigger_mode(self, exposure=200000): #exposure 200000'e baştan ayarlı
    #trigger modda kamerayı başlatır.
    self.mode = "trigger" #kamera modunu trigger'a ayarla
    ips = HikCamera.get_all_ips() #kamera IP'lerini listele
    if not ips: #eğer IP bulunamazsa kamera bulunamadı hatası ver
        print("Kamera bulunamadı.")
        return False
    self.ip = ips[0] #ips'in ilk elemanı self.ip'ye atanır
    print(f"Kamera IP: {self.ip}") #atanan IP ekrana basılır
    self.cam = HikCamera(ip=self.ip) #cam nesnesine HikCamera fonksiyonuna atanan self.ip verilerek camera nesnesi atanır

    try:
        self.cam.__enter__() #cameraya gir
        self.cam["TriggerMode"] = "On" #trigger mode on ayarlı
        self.cam["TriggerSource"] = "Line0" #butondan trigger bekliyor
        self.cam["TriggerActivation"] = "RisingEdge" #yükselen kenar 0'dan 1'e geçiş anında tetikleniyor
        self.cam["LineSelector"] = "Line0" #hat seçimi
        self.cam["ExposureAuto"] = "Off" #otomatik exposure kapalı
        self.cam["ExposureTime"] = exposure #setup_trigger_mode fonksiyonu tanımlanırken verilen exposure değeri verilmiş
```

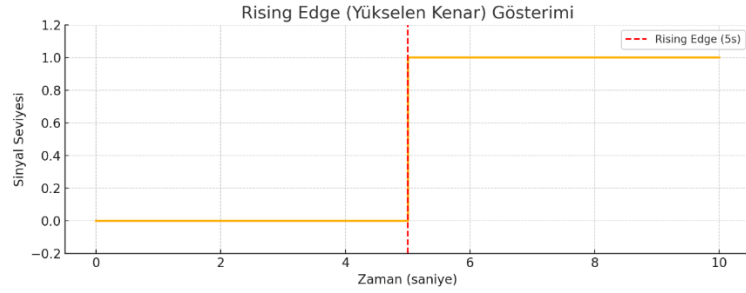
Şekil 4.12. Tetikleme modu kod bloğu

setup_trigger_mode(): Bu fonksiyon HIKRobot kameranın **harici tetikleme (trigger)** modunda çalışmasını sağlayacak şekilde yapılandırılmasını sağlar. Bu modda kamera, sadece dış dünyadan gelen bir sinyalle (örneğin butona basıldığında) görüntü alır. Line0 hattı üzerinden 0V'tan 12V'a geçiş (Rising Edge) algılandığında kamera tek kare çekim yapar. Otomatik pozlama kapatılmış ve pozlama süresi manuel olarak tanımlanmıştır. Bu yapı, sistemin tetik sinyaliyle senkronize çalışmasını sağlar.

- def setup_trigger_mode: kamerayı tetikleme moduna alan genel fonksiyon
- self.mode = "trigger": kamerayı tetikleme moduna alır
- HikCamera.get_all_ips() : bağlı kameraların IP'lerini tarar
- self.ip = ips[0] ilk indeksteki IP'yi ip değişkenine atar
- self.cam = HikCamera(ip=self.ip): atanan IP'yi kullanarak cam nesnesi oluşturur.
- self.cam["TriggerMode"] = "On": "cam" nesnesini trigger mode alır. Kamera sürekli görüntü almak yerine kaynaktan tetik bekler.

- self.cam["TriggerSource"] = "Line0" Tetikleme kaynağı olarak fiziksel giriş hattı belirlenir. Bu sayede butona basıldığında kamera tetiklenir. 6-pin Hirose konnektöründeki Pin 3 - Opto- Isolated Input +(IN+) kısmına 12V verilerek tetikleme sağlanır.
- self.cam["TriggerActivation"] = "RisingEdge" Rising Edge elektronikte önemli bir terimdir. Bir sinyalin 0'dan 1'e(0'dan 12V'a) yükseldiği an olarak tanımlanabilir. Butona basıldığında Rising Edge anında kamera tetiklenir.

Kamera tetiklenir, ardından işlem sonlanır.



Şekil 4.13. Yükselen kenar gösterimi

- self.cam["LineSelector"] = "Line0" komutu, kameranın hangi fiziksel hattının yapılandırılacağını belirtir. Bu satır sayesinde, kamera tetikleme sinyali olarak kullanılacak olan Line0 hattını tanıır ve ilgili ayarlar bu hat üzerinde uygulanır. Özellikle harici tetikleme uygulamalarında, sistemin doğru hat üzerinden tetikleme alabilmesi için bu komutun tanımlanması gereklidir. status = self.cam["LineStatus"] kodu yürütüldüğünde self.cam["LineSelector"] = "Line0" kodu yürütülmüş olduğundan Line0'daki durum status değişkenine atanır.
- self.cam["ExposureAuto"] = "Off" otomatik pozlamayı kapatır
- self.cam["ExposureTime"] = exposure fonksiyon tanımlanırken verilen exposure değerini pozlama süresi olarak ayarlar.

```

36 try:
37     self.cam["AcquisitionStart"] = "" #kamera görüntü almaya başla
38     print("Kamera streaming başlatıldı.")
39 except:
40     print("AcquisitionStart başlatılamaz, devam ediliyor.")
41
42 return True
43 except Exception as e:
44     print("Kamera kurulurken hata:", e)
45     return False
46
47 #alttaki fonksiyon sürekli kare üreten freerun mode'un fonksiyonudur
48 def setup_freerun_mode(self, exposure=200000): #tetik beklemeden sürekli kare üretir.
49     self.mode = "freerun" #mode freerun ayarlanır
50     ips = HikCamera.get_all_ips() #kamera IP'lerini listeler
51     if not ips: #IP bulunmazsa hata verir
52         print("Kamera bulunamadı.")
53         return False
54     self.ip = ips[0] #ilk indeksdeki IP'yi self.ip değişkenine atar
55     print(f"Kamera IP: {self.ip}") #self.IP'yi ekrana basar
56     self.cam = HikCamera(ip=self.ip) #self.cam nesnesine HikCamera'dan ilgili IP'ye sahip kamerayı atar
57
58 try:
59     self.cam._enter__() #kameraya gir
60     self.cam["TriggerMode"] = "Off" #bu fonksiyon triggermodda değil de freerun modda olduğu için "TriggerMode" kapalı
61     self.cam["ExposureAuto"] = "Off" #otomatik exposure kapalı
62     self.cam["ExposureTime"] = exposure #exposure time fonksiyon atanırken zaten tanımlanıyor
63
64 try:
65     self.cam["AcquisitionStart"] = "" #kamera görüntü almasını başlat
66     print("Kamera streaming başlatıldı.")
67 except:
68     print("AcquisitionStart başlatılamaz, devam ediliyor.")
69
70 return True
71 except Exception as e:
72     print("Kamera kurulurken hata:", e)
73     return False
74

```

Şekil 4.14. Kamera kontrol sürekli çekim(freerun) kod bloğu

- self.cam["AcquisitionStart"] = "" Kameraya görüntü almaya başlama komutunu verir. Kamera tetikleme modundaysa tetik sinyali geldiğinde fotoğraf çeker, freerun modundaysa sürekli görüntü göndermeye başlar.
- def setup_freerun_mode: kamerayı tetik sinyali beklemeden sürekli görüntü üretecek moda sokan fonksiyon.

setup_freerun_mode(): Bu fonksiyon HIKRobot kameranın herhangi bir tetikleme sinyali beklemeden sürekli görüntü üretmesini sağlamak amacıyla yapılandırılmıştır. "TriggerMode" parametresi "Off" yapılarak tetikleme kapatılmış, "ExposureAuto" devre dışı bırakılarak pozlama süresi "ExposureTime" parametresi ile manuel olarak ayarlanmıştır. Kamera "AcquisitionStart" komutu ile sürekli kare üretmeye başlar ve alınan görüntüler daha sonra işlenmek üzere kullanılabilir.

```

74
75 def wait_for_trigger(self):
76     """Sadece tetiklemeli mod için tetik bekler."""
77     if self.mode != "trigger":
78         return True # freerun modda tetik beklenmez
79
80     print("Tetikleme bekleniyor...")
81     previous = False
82     while True:
83         try:
84             self.cam["LineSelector"] = "Line0" #LineSelector butondan tetik gelmesini denetler
85             status = self.cam["LineStatus"] #status'a LineStatus'u atar
86         except Exception as e:
87             print("LineStatus okunamadı:", e)
88             return False
89
90         if not previous and status: #previous false ve status True ise if True döner
91             print("Tetik algılandı.")
92             time.sleep(0.3)
93             return True
94
95     previous = status
96     time.sleep(0.01)

```

Şekil 4.15. Kamera kontrol tetik bekleme kod bloğu

wait_for_trigger() fonksiyonu, kamera sisteminde yalnızca "trigger" modundayken çalışacak şekilde tasarlanmıştır. Fonksiyon, "LineStatus" parametresi ile Line0 hattına uygulanan sinyali izler. False durumdan True'ya geçiş (yani Rising Edge) algılandığında, kamera bir tetikleme sinyali aldığını kabul ederek işlemi başlatır. Bu yapı sayesinde sistem fiziksel bir tetikleme (örneğin buton basımı) ile senkronize olarak görüntü alımı gerçekleştirilebilir.

```

97
98 #alttaki fonksiyon kameradan frame olarak "frame" adlı değişkene bu çerçeveyi/görüntüyü atar.
99 #klasördeki dosya isimlerini kontrol eder
100 #kontrol ettikten sonra klasörde olmayan ilk isimle dosyayı kaydeder.
101 #ayrıca bu frame değişkenini döndürür.
102 def get_frame(self, save=True):
103     """Görüntü al, isterse kaydeder."""
104     try:
105         frame = self.cam.get_frame() #kameradan görüntü al frame değişkenine ata
106         if frame is None:
107             print("UYARI: Kamera frame döndürmedi (None geldi).")
108             return None
109
110         print("Görüntü alındı.")
111
112         #klasik dosya/klasör kontrolü ve kaydetme
113         if save:
114             os.makedirs("output", exist_ok=True)
115             i = 1
116             while True:
117                 filename = f"output/kayit_{i:03}.jpg"
118                 if not os.path.exists(filename):
119                     break
120                 i += 1
121             cv2.imwrite(filename, frame)
122             print(f"📷 Görüntü kaydedildi: {filename}")
123
124         return frame
125
126     except Exception:
127         print("Görüntü alırken hata:")
128         traceback.print_exc()
129         return None
130
131 #alttaki shutdown fonksiyonu kamerayı kapatır.
132 def shutdown(self):
133     """Kamerayı düzgün şekilde kapatı-.."""
134     try:
135         if self.mode in ["trigger", "freerun"]: #eğer mode trigger veya freerun modunda ise
136             self.cam["AcquisitionStop"] = "" #kamera akışını durdur
137     except:
138         pass
139     if self.cam: #kamera nesnesi True ise
140         self.cam.__exit__(None, None, None) #çık
141         print("Kamera kapatıldı.")
142

```

Şekil 4.16. Kamera kontrol fotoğraf çekme ve kamera kapatma kod bloğu

get_frame(): Bu fonksiyon kameradan alınan bir görüntü karesini (frame) open cv kütüphanesi aracılığıyla elde eder. Görüntü isteğe bağlı olarak diske .jpg formatında kaydedilir. Kayıt işlemi sırasında, var olan dosyalar kontrol edilerek aynı isimle üzerine yazılmaması sağlanır. Böylece her kare, sıralı olarak ayrı bir dosya şeklinde kaydedilir.

shutdown(): Bu fonksiyon kameranın çalışmasını güvenli bir şekilde durduran yapıdır. Kamera moduna göre görüntü alımı durdurulur (AcquisitionStop) ve ardından sistemden bağlantısı kesilerek kaynaklar serbest bırakılır. Bu işlem sistemin kararlılığını sağlamak ve bir sonraki çalıştırmada bağlantı hatalarının önüne geçmek için önemlidir.

BÖLÜM 5. KARMAŞIK MÜHENDİSLİK UYGULAMASI

5.1. Genel Bilgiler

Karmaşık mühendislik uygulaması kapsamında şirketin hâlihazırda devam eden Tübitak projesi üzerinde çalışılmıştır. Bu projenin çeşitli kısımlarında eklemeler ve düzenlemeler yapılmıştır. Bu proje daha önceden başlamış olduğundan ötürü ortada sıfırdan bir ürün çıkarmaktan ziyade var olan koda/algoritmaya entegre olma ve üzerinde ekleme/değişiklik yapma işlemleri gerçekleştirilmiştir. Bu projede birden fazla kişinin çalışması nedeniyle görev dağılımı yapılmış buna göre projenin ilgili kısımları üzerinde çalışılmıştır.

Çalışmanın çeşitli aşamaları/parçaları vardır. Bunlara genel olarak değinilecek olsa da şirket sırrı olmasından ötürü çok detaylı bir anlatım yapılması mümkün değildir. Bu çalışmanın amacı camların boyutlarının ölçülmesi ve çiziklerin tespit edilmesidir. Staj boyunca daha çok cam boyutlarının ölçümü ve bununla bağlantılı konularla ilgilenilmiştir.

Cam ölçümü tarafında şu konular vardır:

- Kamera ile haberleşme
- Kamera kalibrasyonu
 - Kamera kalibrasyonu için stabil ortamın kurulması
 - Kamera kalibrasyonu için çeşitli yöntemlerin geliştirilmesi ve uygulanması
- Python Pymodbus üzerinden PLC ile haberleşme ve konveyör kontrolü
 - Lazer sensörden alınan veri ile PLC'ye gönderilen sinyalin kullanılması
- PyQt ile arayüz yazılımı
- CAD dosyası ile cam ölçümünün kıyaslanması

Proje kapsamında çeşitli görevler üstlenilmiştir. Farklı zamanlarda projenin farklı parçalarında geliştirmeler ve eklemeler yapılmıştır.

Proje Python ve C++ dillerinde yazılmıştır. Projenin görüntü işleme ile ilgili kısımlarında C++ kullanılmaktadır. Kalan işlerde Python kullanılmaktadır. Arayüz için PyQt, PLC ile haberleşme için pymodbus kütüphanesi, dosyanın ana iskeletini oluşturan main.py dosyası, Hikvision kameralarla bağlantıda kullanılan hik_camera.py ve daha birçok dosyada Python programlama dili kullanılmıştır.

5.2. Yapılanlar

Pymodbus kütüphanesi aracılığıyla Modbus TCP üzerinden PLC ile haberleşilmiş ve çeşitli işlemler gerçekleştirilmiştir. PLC sistemde konveyörü hareket ettirmek için kullanılmaktadır. Bunun yanı sıra fotoelektrik sensörden gelen sinyaller aracılığı ile PLC'den bilgi alınmakta buna göre komut yürütülmektedir.

Bu proje daha önceden başlamış bir çalışma olduğu için belli bir kod yapısına sahipti. Bu kodlar içerisindeki deltaPlc.py dosyasındaki sınıflar main.py ana dosyasında bir modül olarak kullanılmaktaydı ancak sistem çalışır vaziyette değildi.

deltaPlc.py dosyası main.py dosyasında kullanılan fonksiyonlara sadık kalınarak baştan aşağı elden geçirildi ve çeşitli iyileştirmeler yapıldı. Daha önce PLConnect adıyla tanınan sınıf ismi korundu. Bu sınıf main.py dosyasına “from deltaPlc import PLConnect” şeklinde çağrılmaktaydı.

```
from deltaPlc import PLConnect
```

Şekil 5.1. Python özel modül import edilmesi

Şekil 5.1.'de deltaPlc.py dosyasının bir modül olarak import edilmesi gösterilmiştir. deltaPlc dosyası main.py dosyası içerisinde PLConnect ile GUI üzerinden çalıştırılmaktaydı. Bunun bağımsız testi için deltaPlc.py koduna fonksiyon if __name__ == "__main__" koşulunu sağladığında yani modül olarak değil de ana dosya olarak çalıştırıldığında aktif olacak şekilde CLI test bloğu eklendi.

```
def cli_capture_received():
    print("\n>>> CLI TEST: Capture sinyali alındı! (PLC'den geldiği varsayılıyor) <<<\n")
    plc_controller.capture_signal.connect(cli_capture_received)

def print_cli_menu():
    print("\n--- PLC Kontrol CLI (deltaPlc.py Test) ---")
    print("1. PLC'ye Bağlan")
    print("2. PLC Bağlantısını Kes (connectedPlc)")
    print("3. Servo Konumunu Oku")
    print("4. İleri Manuel Hareket Başlat")
    print("5. Geri Manuel Hareket Başlat")
    print("6. Manuel Hareketi Durdur (move_stop)")
    print("7. Belirli Bir Pozisyona Git (setFromPOS)")
    print("8. Home Yap (setHOME)")
    print("9. Home Sensör Durumunu Oku (sensorHome)")
    print("10. Home Sensörüne Git (setHomeSensor)")
    print("11. Sürekli İşlem Başlat (setStyems - QThread'de)")
    print("12. Sürekli İşlemi Durdur (stop)")
    print("0. Çıkış")
    print("-----")
```

Şekil 5.2. PLC CLI arayüzünün kontrol kısmı

Şekil 5.2.'de CLI arayüzünün işlevlerini kullanıcıya gösteren blok gösterilmektedir. Bu kod ile PLC bağlantısı, servo motorun konumunun okunması, konveyörün hareket ettirilmesi, sensörün durumuna göre hareket edilmesi gibi işlevler eklendi. deltaPlc.py dosyasında değiştirilen bağlantılar main.py dosyası ile ilişkilendirildi ve doğru bağlantılar yapıldı ve bağlantılar düzeltildi.

Ölçüm Scale Haritalaması

CAD Dosyası Otomatik Ölçüm Full Otomatik

CAD Listesi Yükle

Manuel Ölçüm

Fotoğraf Çek Fotoğrafları Sil Ölçüm

Konveyör Hareket Ettir 100 mm

Konveyör Konum Ayarla 200 mm

Konveyör mevcut konum (mm): ---

Konveyör Hareket Ayarları

Hareket Hızı 100

Adım Büyüklüğü 200

Şekil 5.3. Cam ölçüm arayüzü

Şekil 5.3.'te cam ölçümü için kullanılan arayüzden bir kesit gösterilmektedir. main.py dosyasına yeni fonksiyonlar eklenerek GUI üzerinde çalışmayan veya yanlış çalışan bazı buton ve işlevler düzeltildi. “Konveyör hareket ettir” butonu çalışmamaktaydı, “konveyör konum ayarla” butonu da mutlak pozisyon ayarlamak yerine konveyörü görel olarak harekete ettirmekteydi. Öncelikle konveyörü hareket ettir butonu çalışır hale getirildi, ardından konveyör konum ayarla butonu mutlak konum ayarlayacak (görel hareket yerine istenen konuma doğrudan gidecek şekilde) şekilde değiştirildi.

Otomatik ölçüm modunda sensörden alınan bilgiye göre konveyörün hareket etmesi sağlandı.

Full otomatik modda kullanıcıdan .txt formatında bir cad listesi alınması sağlandı. Sensörden gelen sinyale göre cad dosyaları değiştirilerek otomatik ölçüm yapılması sağlandı.

5.3. PLC ile Haberleşme - Arayüz Entegrasyonu

5.3.1. PLC Nedir?

PLC açılımı Programmable Logic Controller olan, endüstriyel otomasyon sistemlerinde makinelerin, üretim hatlarının ve çeşitli proseslerin kontrolünü sağlamak amacıyla kullanılan, mikroişlemci tabanlı bir kontrol cihazıdır. Endüstride yaygın olarak kullanılır. Esnek programlanabilir yapısı sayesinde farklı otomasyon senaryolarına kolaylıkla adapte edilebilir.

PLC'ler elektriksel sinyalleri algılayan giriş birimleri, bu verileri işleyen bir CPU ve çıkış birimlerini içerir. Böylece sahadan(sensör, buton vb.) gelen verileri okuyarak, programlanmış lojik kurallar doğrultusunda kararlar alır ve sonuçları sahaya(motorlar, vanalar, röleler vb.) uygular. Bu sayede üretim hatlarının otomatikleşmesini ve veriminin artmasını sağlar.

PLC'ler çeşitli parçalardan oluşur bunlar şunlardır.

- Giriş birimleri: Sensörler, anahtarlar veya butonlar gibi cihazlardan gelen elektriksel sinyalleri toplar
- İşlemci(CPU): PLC'nin beynidir. Giriş verilerini okur, programlanmış komutlara göre karar verir ve çıkışlara komut gönderir.
- Çıkış birimleri: Motorlar, lambalar, röleler gibi yükleri kontrol eder.
- Hafıza: Kullanıcı tarafından yazılan programları, ayarları ve çalışma verilerini saklar.
- Power supply: PLC'nin çalışması için gerekli DC'yi sağlar.

PLC, bir döngü şeklinde sürekli olarak “girişleri oku - işle - çıkışları kontrol et” şeklinde çalışır. Böylece, işlemin anlık durumu her zaman güncel kalır ve otomasyon ihtiyacına uygun olarak sürekli güncellenir.

PLC'lerin tercih edilmesinde çeşitli avantajlar etkilidir. PLC'ler dayanıklı ve güvenilirdir. Tozlu, titreşimli, sıcaklık dalgalanmalı endüstriyel ortamlarda dahi stabil çalışabilir. Programlama işlemleri kolaydır, ladder diagram gibi kolay dillerle programlanır. Esnekliği yüksektir. Donanım değişimi gerektirmeden yazılım üzerindeki değişikliklerle yeni işlemlere kolayca uyarlanabilir. Bakımı kolaydır, arıza durumunda hızlı tanımlama ve müdahale imkanı sağlar. Bunun yanı sıra modüler bir yapıya sahiptir, giriş/çıkış modülleri eklenerek sisteme yeni özellikler kazandırılabilir.

5.3.2. Modbus

Modbus, 1979 yılında Modicon firması(Schneider Electric) tarafından geliştirilmiş, endüstriyel otomasyon sistemlerinde cihazlar arası veri iletişimi sağlamak amacıyla kullanılan, açık bir iletişim protokolüdür. Özellikle PLC, insan-makine arayüzleri(HMI), sensörler ve benzeri otomasyon bileşenleri arasında veri paylaşımı için yaygın olarak kullanılmaktadır.

Master protokolü, veri alışverişinde Master-Slave(veya Client-Server) modelini benimser. BU modelde “Master” cihaz, iletişimi başlatan ve komut gönderen birimdir. “Slave” cihazlar ise gelen isteklere yanıt veren bağımlı birimlerdir. Bir Modbus ağı, bir Master ve 247 adede kadar Slave cihazı destekleyecek şekilde yapılandırılabilir. Modbus’un veri modeli; coil(dijital çıkışlar), discrete input(dijital girişler), input register(analog girişler) ve holding register(analog çıkışlar) gibi temel veri alanlarından oluşur. Böylece farklı veri tipleri kolaylıkla iletilebilir. Protokolün avantajı arasında basitliği, açık yapısı ve üretici bağımsızlığı yer almaktadır. Bu özellikleri sayesinde farklı markalara ait cihazlar arasında bile sorunsuz entegrasyon sağlanabilir. Ayrıca modbus, hem seri iletişim(RS-232, RS-485) hem de ethernet tabanlı iletişim(Modbus TCP) için uygun çözümler sunar.

5.3.3. TCP

TCP(Transmission Control Protocol), bilgisayar ağlarında güvenilir veri iletimini sağlamak amacıyla kullanılan bir iletişim protokolüdür. TCP, 1970’li yıllarda geliştirilen ve günümüzde internet protokolü(IP) ile birlikte çalışan TCP/IP protokol ailesinin temel bileşenlerinden biridir.

TCP, veri iletiminde bağlantı tabanlı bir yapı sunar. Veri gönderimi öncesinde “üçlü el sıkışma(three-way handshake)” olarak adlandırılan bir bağlantı kurma süreci gerçekleşir. Bu süreç, veri iletiminde güvenilirlik ve sıra bütünlüğü sağlamaktadır. TCP, veri paketlerini sıralama, hata denetimi ve akış kontrolü gibi mekanizmalar içerir. Böylece veri kaybı veya bozulma riskleri minimize edilir. Paketlerin doğru sırayla alıcıya ulaşması ve eksik paketlerin yeniden iletilmesi, TCP’nin temel özelliklerindendir.

Sonuç olarak, TCP protokolü; veri iletiminin doğruluğunu, bütünlüğünü ve sürekliliğini garanti ederek, özellikle kritik uygulamalarda güvenilir bir iletişim altyapısı sunar.

5.3.4. Modbus TCP

Modbus TCP, klasik Modbus protokolünün Ethernet ve IP tabanlı ağlarda çalışan versiyonudur. Geleneksel Modbus’un RTU veya ASCII gibi seri iletişim

protokollerine göre daha modern ve hızlı bir çözüm sunmaktadır. Modbus TCP, TCP/IP protokolü üzerinde Modbus protokolünün veri yapısını taşır ve haberleşme Ethernet ağları üzerinden gerçekleştirilir.

Bu protokolde, Modbus veri modeli(coil, discrete input, input register, holding register gibi) korunurken, iletişimde TCP/IP paketleri kullanılır. Böylece, cihazlar arası veri iletimi LAN(yerel alan ağı) veya WAN(geniş alan ağı) altyapısı üzerinden sağlanabilir.

Modbus TCP seri iletişime göre yüksek iletim hızı sunmaktadır. Uzak mesafelerde güvenilir iletişime sahiptir bunun yanı sıra mevcut ethernet ağlarının kullanımı dolayısıyla kablolu ve altyapı konusunda tasarrufludur.

5.3.5 Pymodbus kütüphanesi

Pymodbus, Python programlama diliyle geliştirilmiş, açık kaynaklı Modbus iletişim kütüphanesidir. Endüstriyel otomasyon sistemlerinde yaygın olarak kullanılan Modbus protokolünün Python ortamında uygulanmasına imkan tanır. Bu kütüphane, hem Modbus RTU(seri tabanlı haberleşme) hem de Modbus TCP/UDP(Ethernet/IP tabanlı haberleşme) protokollerini destekler. Pymodbus, master ve slave modunda çalışabilen yapısıyla, hem kontrol birimi olarak hareket eden Python tabanlı yazılımlar geliştirmeyi hem de Modbus uyumlu cihazlara entegre olmayı mümkün kılar. Ayrıca senkron ve asenkron çalışma modlarına sahip olması farklı uygulama ihtiyaçlarına cevap verebilmesini sağlar.

Bu kütüphane, Python dilini kullanarak, Modbus veri modelindeki coil(dijital çıkış), discrete input(dijital giriş), input register(analog giriş), ve holding register(analog çıkış) gibi veri yapıları ile doğrudan haberleşmeyi kolaylaştırır. Bu özellik, otomasyon projelerinde hızlı prototipleme ve test imkanı sunar.

Pymodbus kütüphanesi, endüstriyel otomasyon sistemlerinin modern yazılım gereksinimlerine hitap eden önemli bir araç olarak değerlendirilmektedir. Python'un platform bağımsızlığı ve geniş kütüphane ekosistemiyle birleşerek, veri toplama, uzaktan izleme, kontrol ve endüstriyel süreç optimizasyonu gibi birçok alanda kullanılabilir. Açık kaynak kodlu yapısı sayesinde, sürekli olarak güncellenmekte ve geniş bir geliştirici topluluğu tarafından desteklenmektedir.

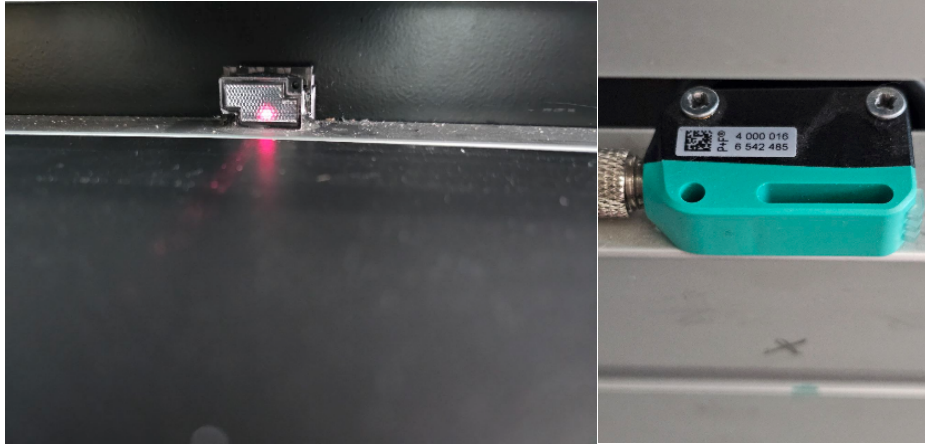
Pymodbus, Python ortamında Modbus protokolü ile veri alışverişi ve kontrol ihtiyaçlarını karşılayan, esnek, genişletilebilir ve modern bir haberleşme çözümü sunar. Endüstriyel otomasyon alanında Python tabanlı projelerde sıklıkla tercih edilen bu kütüphane, mühendislik uygulamaları ve araştırma-geliştirme faaliyetleri için önemli bir araç olarak değerlendirilmektedir.

5.3.6 Retro Reflektif Fotoelektrik Sensör

Bu sensör, ışık tabanlı algılama prensibine sahip bir fotoelektrik algılama sensörü olarak tanımlanabilir. Gövdesinin içerisinde yer alan ışık kaynağı, belirli bir dalga boyunda ışık yayarak çalışır. Sensörün ön yüzünde yer alan optik çıkış, karşısında yer alan yansıtıcı bir yüzeye (reflektör) doğru ışık gönderir. Reflektörden geri yansıyan ışık, sensörün alıcı ünitesi tarafından algılanır.

Algılama prensibi, gönderilen ışığın geri yansımalarının kontrol edilmesine dayanır. Işık yolu herhangi bir nesne tarafından kesildiğinde, alıcı ünite bu değişimi tespit eder ve sensörün çıkış birimi tetiklenir. Böylece sensör, nesnenin algılanmasını sağlayarak dijital bir sinyal üretir.

Sensör, retro-reflektif algılama prensibine uygun olarak çalışır ve bu sayede yansıtıcı bir yüzey yardımıyla ışık yolunu tamamlar. Karşılıklı verici ve alıcı ünitelerin sensör gövdesi içinde kompakt biçimde bulunması, kurulum süresini ve ayar gereksinimini önemli ölçüde azaltır. Bu özellikleriyle sensör, üretim hatlarında parça var/yok kontrolü, pozisyon tespiti ve hızlı tetikleme gibi endüstriyel uygulamalarda sıkça tercih edilmektedir. Ayrıca endüstriyel otomasyon sistemlerinde PLC gibi kontrol birimleriyle kolay entegrasyon sağlayarak işlem güvenilirliğini artırır.



Şekil 5.4. Retro reflektif fotoelektrik sensör

5.3.7 Yapılan İşlemler

PLC'ye veri yazma ve PLC'den veri okuma işlemlerinde bazı yardımcı fonksiyonlar kullanılmaktadır.

```
def convert_to_signed(value, bits=32):
    if value >= (1 << (bits - 1)):
        value -= (1 << bits)
    return value

def write_32bit_value(client, address, value):
    low_word = value & 0xFFFF
    high_word = (value >> 16) & 0xFFFF

    result_low = client.write_register(address, low_word)
    if result_low.isError():
        print(f"HATA (write_32bit_value): 32-bit düşük word yazılamadı (Adres: {address}): {result_low}")
        return False

    # time.sleep(0.02)

    result_high = client.write_register(address + 1, high_word)
    if result_high.isError():
        print(f"HATA (write_32bit_value): 32-bit yüksek word yazılamadı (Adres: {address+1}): {result_high}")
        return False

    # print(f"PLC'ye 32-bit yazıldı: Adres={address}, Değer={value} (Low: {low_word}, High: {high_word})")
    return True

def get_bit_from_word(word, bit_position):
    return (word >> bit_position) & 1
```

Şekil 5.5. PLC kontrolünde Modbus yardımcı fonksiyonları

Şekil 5.4.'te bu yardımcı fonksiyonlar gösterilmektedir. Bu üç yardımcı fonksiyon, Python scripti ile PLC arasındaki veri dönüşümlerini ve kontrolü sağlar. convert_tosigned fonksiyonu, PLC'den okunan 32-bit işaretli verileri, ikinin tümleyeni kuralıyla işaretli tam sayılara dönüştürür. İkinci fonksiyon olan write_32bit_value fonksiyonu 32-bitlik bir veriyi iki ayrı 16 bit register'a bölerek Modbus TCP/IP protokolü ile PLC'ye yazar. Son fonksiyon olan get_bit_from_word fonksiyonu PLC'nin tuttuğu bir 16 bit sayının içindeki belli bir bitin 1 mi yoksa 0 olduğunu bulur. Örnek olarak sensörde nesne var, nesne yok bilgisini almak bir örnektir.

```
class PLConnect(QThread): # QThread'den miras alıyorsa
    capture_signal = pyqtSignal()
    auto_measure_signal = pyqtSignal()
    full_measure_signal = pyqtSignal()

    def __init__(self, path="config/config.json"): # main.py'deki path'e uygun varsayılan
```

Şekil 5.6. PyQt thread işlemleri için QThread sınıfından miras alma

Şekil 5.5.'te thread işlemleri için kullanılan kod bloğu gösterilmektedir. PLConnect sınıfı, PyQt5 arayüzü ile PLC arasında asenkron haberleşmeyi sağlamak amacıyla QThread sınıfından türetilmiştir. Bu yapı, GUI'nin kullanıcı etkileşimleri sırasında kesintiye uğramadan PLC verilerini almasına imkan tanır. Buradan 3 tane sinyal alınmaktadır. İlk sıradaki capture_signal ile kamera çekim sinyali alınırken, auto_measure_signal ile GUI'deki otomatik ölçüm butonunun main.py dosyasında yürüttüğü fonksiyonlar için, full_measure_signal ile de GUI'deki full otomatik ölçüm için sinyal alınmıştır. Bu sinyaller PyQt ile oluşturulan GUI ile dosyalar arasında bağlantı kurmak için kullanılmaktadır.

Init fonksiyonu tanımlanırken path =”config/config.json” şeklinde belirtilen dosya PLC’nin temel ayarlarını içerir. Bu ayarlar arasında PLC’nin IP adresi, port gibi temel bağlantıların yanı sıra motor kontrolü, home(referans) pozisyonlama, manuel hareket gibi çeşitli register adreslerini içerir.

```
try:
    with open(path, "r") as f:
        self.config = json.load(f)
```

Şekil 5.7. JSON dosyasının yüklenmesi

Bu dosya json formatındadır ve kullanılmak üzere config değişkenine atanır.

```
"REGISTER_ADDRESS":6008,
"FROM_POS_Speed":20020,
"FROM_POS_DISTANCE":20016,
"FROM_POS_SET":317,
```

Şekil 5.8. PLC config JSON dosyasından bazı register adresleri

Şekil 5.7.’de PLC register adreslerinin bir kısmı gösterilmektedir. Bu değerler PLC’ye veri yazmak ve PLC’den veri okumak için kullanılan register adresleridir. Register, bir PLC’nin veya bir dijital cihazın hafızasında yer alan küçük veri saklama birimidir. Modbus haberleşmesinde, her register bir sayı saklar. Örnek olarak bu motorun hızı olabilir veya sensör değeri olabilir.)

```
self.FROM_POS_SET = self.config.get("FROM_POS_SET")
self.FROM_POS_DISTANCE = self.config.get("FROM_POS_DISTANCE")
self.FROM_HOME_SENSOR = self.config.get("FROM_HOME_SENSOR")
self.HOME_POSE = self.config.get("HOME_POSE")
self.GO_HOME = self.config.get("GO_HOME")
self.HOME_SPEED = self.config.get("HOME_SPEED")
```

Şekil 5.9. JSON dosyasından adreslerin alınması

Şekil 5.8.’de JSON dosyasından adresleri alan kodlardan bir parça göBu kısımda JSON dosyasından ilgili adresler alınmakta ve değişkenlere atanmaktadır. Bu atanan değişkenlerle kodun ilerleyen kısımlarında çeşitli işlemler gerçekleştirilmektedir.

```

def setHOME(self, distance, speed):
    if not self.connect_status or not self.client:
        print("setHOME: PLC bağlantısı yok.")
        return False

    print(f"HOME {distance} noktasına {speed} hızıyla gidiliyor...")
    rr_speed = self.client.write_register(self.HOME_SPEED, int(speed * 10))
    if rr_speed.isError():
        print(f"HATA (setHOME): Home hızı yazılamadı (Adres {self.HOME_SPEED}): {rr_speed}")
        return False

    if not write_32bit_value(self.client, self.HOME_POSE, int(distance * 1_000_000)):
        return False

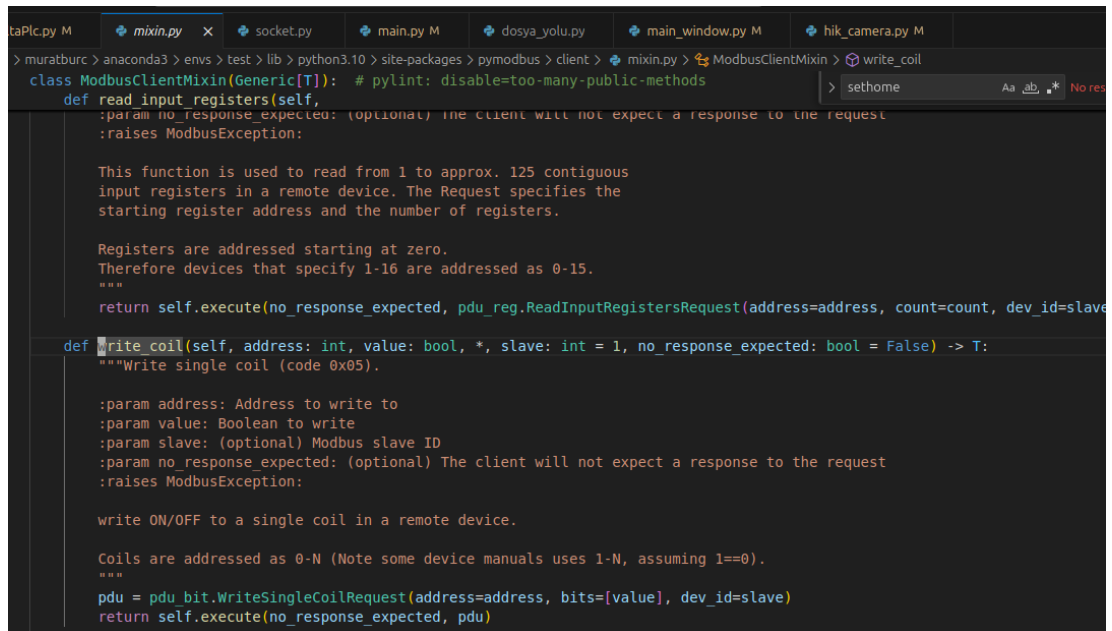
    time.sleep(0.1)
    rr_go = self.client.write_coil(self.GO_HOME, True)
    if rr_go.isError():
        print(f"HATA (setHOME): Home biti set edilemedi (Adres {self.GO_HOME}): {rr_go}")
        return False

    print("setHOME: Komut gönderildi.")
    return True

```

Şekil 5.10. PLC için kullanılan fonksiyonlardan bir örnek

Şekil 5.9.'da PLC ile konveyör kontrolü için kullanılan fonksiyonlardan biri gösterilmektedir. setHome fonksiyonu ile konveyörün referans noktasına gitmesi komutu verilmektedir. Fonksiyon, öncelikle PLC bağlantı durumunu kontrol eder. Ardından, PLC registerlarına home dönüşü için gereken hız ve mesafe bilgilerini yazar. Hız verisi 16 bit olarak write_register komutu ile yazılırken, mesafe verisi 32 bit olduğu için write_32bit_value fonksiyonu kullanılarak iki registera bölünerek gönderilir. Yazma işlemlerinin ardından 0.1 saniyelik kısa bir bekleme süresi eklenir. Son olarak write_coil fonksiyonu aracılığıyla home dönüşünü başlatan bit ayarlanır.



```

class ModbusClientMixin(Generic[T]): # pylint: disable=too-many-public-methods
    def read_input_registers(self,
        :param no_response_expected: (optional) The client will not expect a response to the request
        :raises ModbusException:

        This function is used to read from 1 to approx. 125 contiguous
        input registers in a remote device. The Request specifies the
        starting register address and the number of registers.

        Registers are addressed starting at zero.
        Therefore devices that specify 1-16 are addressed as 0-15.
        """
        return self.execute(no_response_expected, pdu_reg.ReadInputRegistersRequest(address=address, count=count, dev_id=slave

    def write_coil(self, address: int, value: bool, *, slave: int = 1, no_response_expected: bool = False) -> T:
        """Write single coil (code 0x05).

        :param address: Address to write to
        :param value: Boolean to write
        :param slave: (optional) Modbus slave ID
        :param no_response_expected: (optional) The client will not expect a response to the request
        :raises ModbusException:

        write ON/OFF to a single coil in a remote device.

        Coils are addressed as 0-N (Note some device manuals uses 1-N, assuming 1==0).
        """
        pdu = pdu_bit.WriteSingleCoilRequest(address=address, bits=[value], dev_id=slave)
        return self.execute(no_response_expected, pdu)

```

Şekil 5.11. Pymodbus kütüphanesi write_coil fonksiyonu

Şekil 5.10.'da pymodbus fonksiyonlarından biri gösterilmektedir. write_coil fonksiyonu pymodbus kütüphanesinin içinden gelen bir fonksiyondur. Bir modbus cihazına tek bir coil(bit) değeri yazılabilmesini sağlayan bir fonksiyondur. Bu fonksiyon, modbus protokolünün 0x05(Write single coil) fonksiyon kodunu kullanarak, cihazın coil olarak adlandırılan dijital çıkışlarına doğrudan komut gönderir.

Fonksiyon, yazılacak coid adresini (adress), hedef değeri(value: True veya False) veya isteğe bağlı olarak cihazın slave ID'sini(slave) parametre olarak alır.

write_coil fonksiyonu, endüstriyel kontrol ve otomasyon sistemlerinde pompa, motor, vana gibi dijital çıkışların kontrolü için yaygın olarak kullanılır. Böylece Python uygulaması, PLC veya başka bir Modbus cihazı üzerinde doğrudan ON/OFF kontrolü gerçekleştirebilir.

PLC'nin kontrolü için kullanılan deltaPlc.py dosyasının içerisinde setHOME fonksiyonuna benzer fonksiyonlar vardır. Bu fonksiyonlar deltaPlc.py dosyasındaki class PLConnect sınıfında bulunmaktadır. Bu sınıfın içindeki fonksiyonlar main.py dosyasının içindeki çeşitli fonksiyonlarda kullanılmakta, çeşitli senaryolara göre PLC'ye sinyal gönderilmektedir.

5.7.4 Full Otomatik Ölçüm Modu - CAD Listesi ve Sensör Haberleşmesi

Arayüzde full otomatik ölçüm şeklinde bir mod vardır. Bu işlemde kullanılacak elemanlar şunlardır.

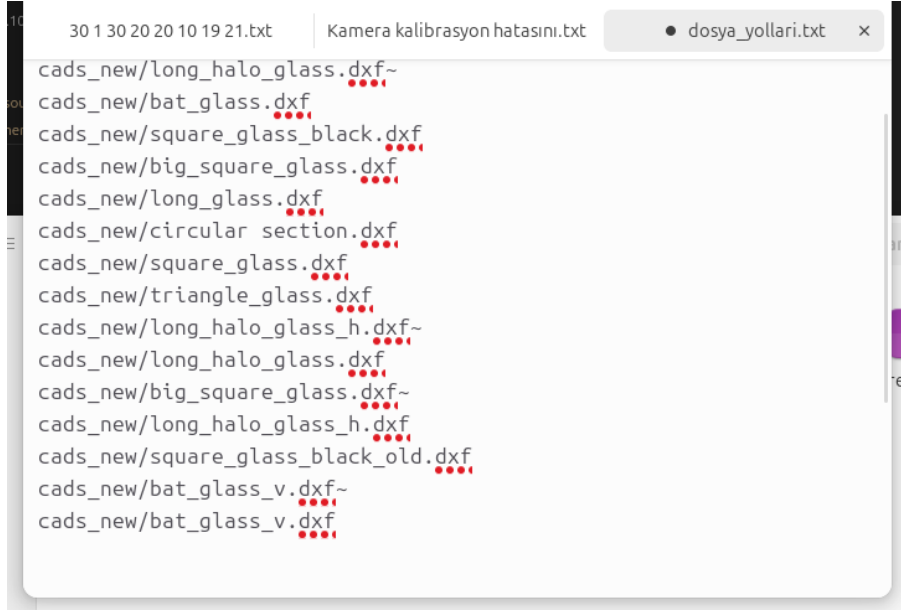
- PLC
- Kamera
- Sensör
- Konveyör
- Bilgisayar
- Kullanıcı arayüzü
- Ölçülecek nesne

Konveyör üzerinde reflektif fotoelektrik sensör bulunmaktadır. Bu sensör önüne nesne geldiğinde PLC'ye sinyal vermektedir. Bu sensör kullanılarak full otomatik ölçüm yapılması istenmektedir.

Olay örgüsü şu şekilde olmaktadır.

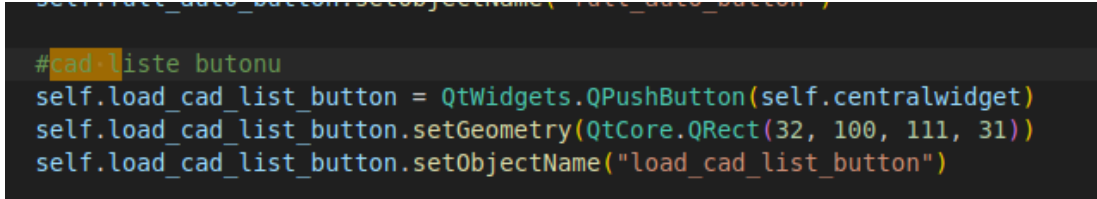
- .txt formatında dosyaya cad listesinin yollarını ver ve ardından ilk dosyayı yükle.
- Sensörün önüne nesne gelesiye kadar konveyörü aralıksız hareket ettir.
- Sensör önüne nesne geldiğinde konveyörü durdur.

- Fotoğraf çek ardından arayüzden ayarlanan miktar kadar konveyörü hareket ettir.
- Tekrar fotoğraf çek ve konveyörü tekrar hareket ettir.
- Nesne sensör önünden çekildikten sonra .txt dosyasından bir sonraki CAD dosyasına geç. Ayrıca bir sonraki nesne sensör önüne gelesiyeye dek konveyörü aralıksız şekilde ilerlet.



Şekil 5.12. CAD dosya yolları

Şekil 5.11.'de CAD dosyalarının listesi gösterilmektedir. Sensörden nesne her kaybolduğunda bir sonraki yola geçecektir.



Şekil 5.13. PyQt CAD listesi yükle butonu

Şekil 5.12.'de görünen kodlarla CAD listesi yükleme butonu GUI arayüzüne eklenmiştir. Bu kodların bulunduğu dosya PyQt arayüz bileşenlerini barındıran main_window.py dosyasıdır.

```

def retranslateUi(self, MainWindow):
    _translate = QtCore.QCoreApplication.translate
    MainWindow.setWindowTitle(_translate("MainWindow", "MainWindow"))

    self.tabWidget.setTabText(self.tabWidget.indexOf(self.tab), _translate("MainWindow", "Ölçüm"))

    self.cad_button.setText(_translate("MainWindow", "CAD Dosyası"))
    self.auto_measure_button.setText(_translate("MainWindow", "Otomatik Ölçüm"))
    self.full_auto_button.setText(_translate("MainWindow", "Full Otomatik"))

    self.load_cad_list_button.setText(_translate("MainWindow", "CAD Listesi Yükle")) #cad listesi yükle butonu isimlendirmesi

```

Şekil 5.14. retranslateUi fonksiyonu ile buton isimlendirmesi

Şekil 5.13.2te def retranslateUi fonksiyonunun şekilde görülen son satırı ile bu butonun isimlendirilmesi ve MainWindow sınıfı ile bağlantısı yapılmıştır.

```

# gui_sourceda bulunan ana pencere tasarımını istenilen işlemlere bağlayan class.
class MainWindow(QMainWindow):
    cad_path=None
    res_win=None

    def __init__(self,camera_configs:list[str]):
        super(MainWindow,self).__init__()

        self.ui=Ui_MainWindow()
        self.ui.setupUi(self)

        #cad listesi yükle liste/index
        self.cad_list = []
        self.current_cad_index = 0

```

Şekil 5.15. CAD listesi için kullanılacak değişkenler

Şekil 5.14.’teki main.py dosyasından bir kesittir. main.py dosyasında bulunan class MainWindow sınıfında cad listesi ve indexi için şekilde son satırda görülen değişkenler oluşturulmuştur. Değerler bu değişkenlerde tutulacaktır.

```

self.ui.load_cad_list_button.clicked.connect(self.load_cad_list) #Cad Liste Yükle bağlantısı

```

Şekil 5.16.Arayüzdeki buton ile fonksiyonun bağlanması

Şekil 5.15.’te görülen kod satırı CAD listesi yükle butonuna tıklandığında load_cad_list metodunun çağrılmasını sağlar. self.ui.load_cad_list_button.clicked kısmı kullanıcı butona tıkladığında yayılan sinyali ifade eder, “.connect(self.load_cad_list)” kısmı ise o sinyal alındığında çalışacak fonksiyon olarak load_cad_list fonksiyonunu işaret eder.

```

#sensör sinyallerine bağlan - cad listesi yükle
self.plc.full_measure_signal.connect(self._on_sensor_trigger)

```

Şekil 5.17. Full otomatik buton sinyaline fonksiyon bağlanması

Şekil 5.16.’daki kod satırı Full otomatik butonuna basıldığında full_measure_signal sinyalinin tetiklenmesini sağlar bu sinyal tetiklendiğinde _on_sensor_trigger fonksiyonu devreye girer.


```

def load_cad_list(self):
    path, _ = QtWidgets.QFileDialog.getOpenFileName(
        self, "CAD Listesi Yükle", "", "Text Files (*.txt)")
    if not path:
        return

    with open(path, 'r', encoding='utf-8') as f:
        raw = [ln.strip() for ln in f if ln.strip()]

    # Sadece '.dxf' ile biten ve gerçekten var olan yolları al
    valid = []
    skipped = []
    for p in raw:
        if not p.lower().endswith('.dxf'):
            skipped.append(p)
        elif not os.path.isfile(p):
            skipped.append(p)
        else:
            valid.append(p)

    self.cad_list = valid
    self.current_cad_index = 0

    # Kullanıcıya bilgilendirme
    if valid:
        self.set_cad_by_index(0)
        QMessageBox.information(
            self, "CAD Listesi Yüklendi",
            f"{len(valid)} dosya bulundu ve yüklendi.\n"
            f"{len(skipped)} yol atlandı."
        )
    else:
        QMessageBox.warning(
            self, "CAD Listesi Boş",
            "Geçerli bir .dxf dosyası bulunamadı."
        )

    if skipped:
        print("Atlanan yollar (geçersiz veya bulunamadı):")
        for p in skipped:
            print(" ", p)

```

Şekil 5.18. CAD listesi yükleme fonksiyonu

Şekil 5.17.’deki fonksiyon kullanıcıdan .txt formatında dosya seçmesini ekler ardından CAD dosya yollarını self.cad_list değişkenine atar.

Bu fonksiyonun çalışma şekli şu şekildedir.

- GUI’de “ CAD Liste Yükle” butonuna basıldığında dosya seçme penceresi açılır.
- Kullanıcı bir .txt dosyası seçer.
- Seçilen .txt dosyasındaki her satırı okuyup self.cad_list listesine atar.
- İlk CAD dosyasını (self.cad_list[0]) hemen yüklemek için self.set_cad_by_index(0) fonksiyonunu çağırır.

load_cad_list fonksiyonunda kullanılan set_cad_by_index fonksiyonu şu şekildedir.

```
def set_cad_by_index(self, idx):
    """cad_list'ten idx'deki CAD dosyasını camera_thread'e yükler."""
    cad_path = self.cad_list[idx]
    self.cad_path = cad_path
    self.camera_thread.set_cad(cad_path)
    print(f"[CAD] Yüklendi: {cad_path}")
```

Şekil 5.19. load_cad_list fonksiyonu için yardımcı fonksiyon

Şekil 5.18.'de gösterilen fonksiyon verilen indeks numarasına göre self.cad_list listesindeki CAD dosyasını yükler ve ölçüm için hazırlar.

- idx parametresiyle CAD listesinde hangi CAD dosyasının seçileceği belirlenir.
- self.cad_list[idx] kullanılarak CAD yolu alınır.
- self.camera_thread.set_cad(cad_path) çağrılır. Bu görüntü işleme iş parçacağına yeni CAD dosyasını yüklemesini söyler.
-

```
def full_auto_func(self):
    if self.is_running_auto:
        QMessageBox.warning(self, "Uyarı",
            "Kısa otomatik mod açıldıkten tam otomatik başlatılamaz!")
        return

    self.is_running_full = not self.is_running_full
    if self.is_running_full:
        # CAD listesi mutlaka yüklü olmalı
        if not self.cad_list:
            QMessageBox.warning(self, "CAD Listesi Eksik",
                "Önce CAD listesi yükleyin.")
            self.is_running_full = False
            return

        # İndeksi başa al ama zaten ilk CAD yüklü durumda
        self.current_cad_index = 0
        self.set_cad_by_index(0)

        if self.plc.connect_status:
            dist = int(self.ui.move_conveyor_size_sbox.text())
            spd = int(self.ui.movement_speed_sbox.text())
            self.ui.full_auto_button.setStyleSheet(
                "background-color: red; color: white; font-size:16px;")
            self.plc.distance = dist
            self.plc.speed = spd
            self.plc.full_auto_mode = True
            self.plc.start()
        else:
            self.is_running_full = False
            QMessageBox.warning(self, "PLC Bağlantısı Yok",
                "Lütfen önce PLC'ye bağlanın.")
    else:
        # Tam otomatığı kapat
        self.ui.full_auto_button.setStyleSheet(
            "background-color: green; color: white; font-size:16px;")
        self.plc.stop()
        self.plc.wait()
```

Şekil 5.20. Full otomatik ölçüm fonksiyonu

Şekil 5.19.'da full otomatik ölçümün fonksiyonu görülmektedir.

- Öncelikle tek sefer ölçüm yapan otomatik ölçüm modunun aktif olup olmadığını kontrol eder. Otomatik ölçüm modu bir nesne sensörün önüne bir kez girip çıktıktan sonra konveyörü durdururken full otomatik modu nesne sensörden çıktıktan sonra bir sonraki nesne sensörün önüne gelesiyeye dek konveyörü aralıksız hareket ettirir.
- self.is_running_full = not self.is_running_full satırı ile fonksiyonun açık mı kapalı mı olduğu bilgisi tutulur.

- if self.plc.connect_status bloğu ile PLC'nin bağlantı durumu kontrol edilir eğer bağlantı başarılı ise GUI'den ilgili kısımdan hız ve mesafe bilgisi alınır. Ayrıca full otomatik butonu kırmızı renge döndürür ve çalışıyor anlamını verir.. Bundan sonra dist ve spd değerleri plc'nin distance ve speed değişkenlerine atanır.

self.plc.start() fonksiyonu deltaPlc.py'deki PLCConnect sınıftan türeyen bir threaddir.

Başlatıldığında

- sensör kontrolü
- sensör sinyali takibi
- sensör tetiklendiğinde fotoğraf çekimi
- sensör sonrası CAD geçişi ve yeni ölçüm döngüsel sürecini başlatır.

```
def _on_sensor_trigger(self):
    """
    PLC'den full_measure_signal geldiğinde,
    eğer tam otomatik moddaysak bir sonraki CAD dosyasını yükle.
    """
    if not getattr(self, "is_running_full", False):
        return

    # Daha önce yüklü CAD tamamlandı, şimdi indeksi artır
    self.current_cad_index += 1

    if self.current_cad_index < len(self.cad_list):
        self.set_cad_by_index(self.current_cad_index)
    else:
        # Liste bitti → tam otomatik mod kapanır
        QMessageBox.information(self, "Bitti",
                                "Tüm CAD dosyaları işlendi. Full-auto mod kapatılıyor.")
        self.is_running_full = False
        self.ui.full_auto_button.setStyleSheet(
            "background-color: green; color: white; font-size:16px;")
        self.plc.stop()
        self.plc.wait()
```

Şekil 5.21. Sinyal tetiklemesine göre CAD değişimi

Şekil 5.20.'de _on_sensor_trigger fonksiyonu görülmektedir. Bu sensör sinyali tetiklemesine göre CAD listesini değiştirir. Full otomatik modda full_measure_signal tetiklemesi geldiğinde bir sonraki cad dosyasına geçiş sağlar. Bunun için self.current_cad_index ile index bir sayı arttırılır.

Fonksiyon ayrıca CAD listesinin sonuna gelindiğinde full otomatik modu kapatır ve kullanıcıya listenin sonuna gelindiğine dair uyarı verir. Bununla birlikte self.plc.stop çağrısıyla konveyörü durdurmak için PLC'ye sinyal gönderir. self.plc.wait çağrısı ise QThread sınıfından türeyen PLCConnect PLC Thread'inin durmasını bekler.

BÖLÜM 6. DİSİPLİNLER ARASI UYGULAMALAR

İşletmede bilgisayar mühendisleri ve bilgisayar mühendisliği yüksek lisans öğrencileri ile ortak çalışmalar yapıldı. Genel olarak staj süresince yapılan çalışmalarda bilgisayar mühendisliği ile örtüşen/kesişen çalışmalardı.

6.1 Karmaşık Mühendislik Uygulamasında Yapılan Çalışmalar

Karmaşık mühendislik uygulamasında bahsedilen proje bilgisayar mühendisleri ile ortak şekilde yürütüldü. Bu sebepten ötürü buradaki çalışmalar genel olarak disiplinler arası çalışma olarak ele alınabilir.

Bu proje dama tahtasının görüntüsü alınması ve çeşitli görüntü işleme teknikleri kullanılarak hata haritası oluşturulması yöntemine dayanmaktadır. Burada oluşturulan hata haritası ile yan yana kameralarla ölçüm yapılırken kameraların hassas hizalanması maliyetinden kaçınılmıştır.

Karmaşık mühendislik uygulaması kapsamında PLC tarafı ve bunun main.py dosyasına entegresi ile uğraşılmış. Az önce bahsedilen konu ve bu konulara benzer durumlarda arayüzde ortak çalışma yapılmıştır. Projenin camdaki çizikleri yapay zeka ile tespit etme tarafı da vardır. Bu tarafta da yer yer otomatik veri etiketlemesi çalışması yapılmış, model eğitimi gibi konularda yer alınmıştır. Başlangıçta da denildiği gibi PLC gibi elektrik elektronik mühendisliğine daha çok hitap edilen konularda görev alınsa da genel olarak yazılım üzerine çalışma yapıldığı için yapılan işlerde bilgisayar mühendislerinin yaptığı işlerle kesişim yaşanmış ve ortaklaşa iş yapılmıştır.

BÖLÜM 7. İŞLETMEDE KARŞILAŞILAN PROBLEMLER VE ÇÖZÜMLER

İşletmede karşılaşılan en büyük zorluklardan birisi aynı proje üzerinde farklı kişilerin çalışması dolayısıyla bu çalışmanın birleştirilmesi zorluğudur. Yapılan değişikliklerin doğru şekilde yedeklenmesi ve dağıtılması önemli konulardan birisidir. Geniş bir dosya yapısı bulunmakta farklı kod dosyaları yer almakta ve farklı kişiler bu dosya üzerinde çalışmaktadır. Örnek olarak main.py dosyası üzerinde PLC için ayrı fonksiyonlar, kamera için ayrı fonksiyonlar, algoritma için farklı fonksiyonlar vardır. Bunun yanı sıra proje sadece Python ile yazılmamıştır projenin içerisinde görüntü işleme ile ilgili konular C++ temellidir. Bu karmaşık projenin üzerinde farklı kişilerin doğru şekilde çalışabilmesi için Git versiyon kontrol sistemi kullanılmış ve diğer kişilerden izole ortamlarda değişiklikler yapılmıştır.

Bunun yanı sıra bir diğer problem/zorluk çalışılan projenin sistem/bilgisayar bağımsız çalışabilmesidir. Sistemde yapılan değişikliklerin herkeste aynı tepkiyi vermesi, bağımlılık hatası alınmaması, stabil ve izole bir ortam kurulması adına Docker ve Docker konteynerleri kullanılmıştır. Bu sayede farklı kişilerin bilgisayarında aynı sonuçlar elde edilebilmiş. Projenin tutarlı sonuçlar verebilmesi sağlanmıştır. Karmaşık mühendislik uygulaması kısmında üzerinde çalıştığımız ölçüm projesine diğer mühendislerin üzerinde daha çok çalıştığı çizik projesinin entegre edilmesi, arayüze hızlıca çizik projesinin entegre edilmesi için Docker konteynerleri kullanılmış ve ölçüm projesi diğer bilgisayarlarda hızlıca ayağa kaldırılmıştır.

Karşılaşılan bir diğer zorluk işletim sisteminin getirdiği zorunluluklardır. Karmaşık mühendislik uygulamasında dahil olduğum proje tamamen Ubuntu 24.04 işletim sistemi üzerinde çalışılmıştır. Bunun sebebi olarak Linux'un yapılan işte daha uyumlu olması, kütüphaneler bakımından daha çok desteklenmesi gibi durumlar sayılabilir.

BÖLÜM 8. SONUÇ ve DEĞERLENDİRME

7+1 İşletmede Mesleki Eğitim kapsamında birçok konuda çalışılmış ve farklı alanlarda deneyimler elde edilmiştir. Bunun yanı sıra bilgisayar mühendisleri, bilgisayar mühendisliği yüksek lisans öğrencisi, elektrik elektronik mühendisliği öğrencisi stajyerler ile ortak çalışmalar yürütülmüştür. Birbirinden farklı çeşitli alanlarda deneyim kazanılmıştır. Model eğitimi, doğru veri etiketleme, otomatik veri etiketleme sistemleri üzerinde çalışılmıştır. Bunun yanı sıra endüstriyel kameraların SDK aracılığıyla kontrolü ve entegrasyonu gibi işlerde yer alınmıştır. Bir diğer taraftan karmaşık mühendislik uygulaması kapsamında Linux deneyimi elde edilmiş, burada karşılaşılan sorunlara çözüm aranmış ve tecrübe elde edilmiştir.

Docker tarafında kaynak kodu güvenliği hakkında çalışmalar yapılmıştır. Docker tarafında yapılan bir diğer çalışmada konteyner yapısı sayesinde izole ortamlar oluşturulması ve bunların farklı bilgisayarlar üzerinde hızlıca ayağa kaldırılmasıdır. Karmaşık mühendislik uygulaması kapsamında çalışılan proje farklı bilgisayarlarda ayağa kaldırılarak çalışabilir hale getirilmiştir.

Bir diğer çalışılan konu PLC ile modbus üzerinden haberleşme, sensörden veri alarak konveyöre hareket verme gibi konulardır. Bunun yanı sıra karmaşık mühendislik uygulamasında bahsedilen projede görüntü işleme ile ilgili konulara giriş yapılmıştır. En önemli konu da disiplinler arası çalışmalar kapsamında diğer mühendisler ile ortak ve verimli çalışma yapılmasıdır. Karmaşık mühendislik uygulamasında bahsedilen proje takım çalışmasını geliştiren, ortak proje yürütme becerilerini geliştiren bir deneyim olmuştur.

Bu 7+1 işletmede mesleki eğitim uygulaması kapsamında çok çeşitli alanlarda deneyim elde ettim ve çeşitli projelere uçtan uca dahil oldum. Şirketin büyüklük olarak startup seviyesinde bir şirket olması belli bir işe odaklı hareket etmekten daha çok her türlü işte çalışma ve deneyim elde etme imkanını sunmuştur/zorunluluğunu getirmiştir. Bu 7+1 İşletmede Mesleki Eğitim uygulamasının sektör tecrübesi bakımından bana değerli katkıları olduğunu düşünmekteyim.